



Deep reinforcement learning for robust robot navigation in complex and crowded environments

Jin Meng¹ · Jiahui Zou¹ · Shifeng Wang^{1,2} · Ruibing Yang^{1,2} · Aakash Kumar^{1,2} · Jonghyuk Kim³

Received: 30 June 2025 / Accepted: 22 October 2025 / Published online: 25 November 2025
© The Author(s) 2025

Abstract

In complex environments with dense pedestrian traffic, mobile robots often experience errors and instability during trajectory tracking and dynamic obstacle avoidance tasks. This paper presents a scene perception and decision-making strategy combined with deep reinforcement learning. Temporal sequences of LiDAR data and sub-goal were used as input, and action output is generated via an end-to-end network. We designed an adaptive heading reward that guides the robot to proactively avoid pedestrians while efficiently moving toward its target. Through continuous interaction with a dynamic environment, the robot learns an optimal decision-making strategy by maximizing cumulative rewards. A series of simulation experiments and real-world validations demonstrate that the proposed strategy achieves an effective balance between collision avoidance and real-time performance in robotic navigation. Furthermore, extensive results confirm that the method remains robust in unfamiliar environments and in varying crowd densities. Finally, tests on a hardware platform indicate that the strategy offers strong stability and adaptability in practical applications, effectively meeting obstacle avoidance requirements and validating its reliability in complex dynamic settings.

Keywords Deep reinforcement learning · Adaptive collision avoidance · Sim-to-real transfer · Dense pedestrian environments

1 Introduction

In daily life, unmanned ground vehicles (UGVs) (Desouza and Kak 2002; Asadi et al. 2020; Juang and Chang 2011;

Sarkar et al. 2022) have expanded from traditional industrial applications to commercial applications. Autonomous delivery robots can navigate freely in open urban areas, avoid obstacles, and transport packages or items to designated destinations. Street-cleaning robots can autonomously plan routes, avoid pedestrians and vehicles, and keep the streets clean. These tasks have time constraints, requiring mobile robots to navigate autonomously and quickly to their destinations in spaces filled with moving people and other static obstacles.

Obstacle avoidance (Borenstein and Koren 1989; Larson et al. 2006; Liu et al. 2024) is widely encountered in fields such as autonomous driving and robotic navigation. These tasks typically require real-time decision making to ensure that the agent can avoid collisions in complex and dynamic environments. Traditional obstacle avoidance methods, which are typically based on predefined rules, sensor data, or graph-based planning, often struggle to maintain stability and real-time responsiveness in densely populated and dynamic environments. Rule-based methods lack the capacity for learning and thus exhibit limited adaptability to complex and rapidly changing scenarios. Reinforcement learning has been proven to be effective in dynamic and

✉ Shifeng Wang
sf.wang@cust.edu.cn

Jin Meng
2022200070@mails.cust.edu.cn

Jiahui Zou
2022003598@mails.cust.edu.cn

Ruibing Yang
yangrb@cust.edu.cn

Aakash Kumar
aakash@cust.edu.cn

Jonghyuk Kim
JKim@nauss.edu.sa

¹ School of Optoelectronic Engineering, Changchun University of Science and Technology, Changchun, China

² Zhongshan Institute of Changchun University of Science and Technology, Changchun, China

³ Naif Arab University for Security Sciences, Riyadh, Saudi Arabia

uncertain environments, particularly for tasks that require autonomous decision making. In obstacle avoidance, deep reinforcement learning (DRL) (Li et al. 2019; Tao and Kim 2024) can continuously improve decision making strategies through interactions with the environment, thus learning to find safe paths in changing obstacle scenarios. Compared to traditional methods, DRL-based obstacle avoidance algorithms (Rehani 2015; Li et al. 2023) demonstrate greater adaptability to handle emerging obstacles, unforeseen environmental changes, and diverse task requirements. However, some deep learning approaches lack adaptive reward mechanisms specifically designed for navigation tasks, resulting in suboptimal trade-offs between collision avoidance and goal-directed behavior.

To overcome the limitations of traditional rule-based or static learning methods that lack generalization in dynamic and unfamiliar environments, this study proposes a DRL-based navigation framework with enhanced scene understanding and decision-making capabilities. Specifically, the goals of this paper are as follows:

- 1) To propose a novel data combination that uses historical LiDAR frames and sub-goals as input data for the control network, and to design a feature extraction network to obtain rich scene information.
- 2) An adaptive navigation heading reward function is proposed to adjust the robot orientation based on the positions of the surrounding obstacles. This function helps the robot effectively improve its obstacle avoidance performance.
- 3) To propose an angular velocity penalty that adaptively smooths turning behavior, enhancing motion stability and safety in dynamic navigation scenarios.
- 4) The efficient multiscale attention module (EAM) is introduced, which helps the robot better understand the scene by focusing on local spatial information, thus improving its perception capabilities in complex environments and improving the accuracy and robustness of decision-making.
- 5) The proposed model can be directly applied to various unseen scenarios, including dynamic indoor and outdoor environments with varying crowd densities, to achieve effective obstacle avoidance without additional training.
- 6) The policy, trained on TurtleBot2 within a single simulated environment, can be seamlessly transferred to multiple hardware platforms of varying specifications without retraining, consistently maintaining stable and efficient obstacle avoidance performance.

The remainder of this paper is organized as follows. Section 2 reviews the related work. Section 3 introduces the theoretical background and the overall network architecture of the proposed algorithm. Section 4 describes the

implementation and evaluation of the proposed algorithm. The simulation results in Section 4.2 verify navigation performance, while the hardware experiments in Section 4.3 highlight the small sim-to-real gap. Section 4.4 also provides an analysis of representative failure cases. Finally, Section 5 concludes the work and outlines future research directions.

2 Related work

Obstacle avoidance in dynamic environments has received widespread attention and research in the field of robotics in recent years. Pedestrian avoidance approaches are typically divided into model-based and model-free categories.

2.1 Model-based methods

Model predictive control (MPC) (Schwenzer et al. 2021; Qin and Badgwell 2003) is a classical obstacle avoidance algorithm that ensures that a robot avoids collisions and satisfies dynamic constraints during motion by optimizing control inputs in real time. Kim et al. (2021) proposed an MPC algorithm that uses a nonuniformly spaced and time-varying prediction horizon for path tracking and obstacle avoidance. This method enables fine-grained avoidance decisions for nearby obstacles while providing early warnings for distant ones. Nguyen et al. (2024) modeled the interaction between a robot and pedestrians, allowing the MPC algorithm to make real-time obstacle avoidance decisions based on predicted trajectories. They also introduced a critical safety control framework that integrates nonlinear model predictive control (NMPC) with a noise-aware data-driven prediction model. Using the sparse identification of nonlinear dynamics (SIND) method for state prediction and incorporating control barrier function (CBF) constraints to guaranty safety, the proposed closed-loop system achieves input-to-state stability (ISS) with respect to prediction uncertainties.

In addition, optimization-based methods are effective solutions for dynamic obstacle avoidance. These approaches typically formulate an objective function that considers the relative velocity and distance between the robot and dynamic obstacles by performing global or local path optimizations. Alaliyat et al. (2019) proposed a path planning algorithm based on particle swarm optimization (PSO), which enables the autonomous robot to move smoothly and naturally from the starting point to the goal while ensuring safety for both the robot and its surrounding environment. The algorithm also incorporates three evaluation criteria: energy consumption, travel time, and collision-free path length.

Model-based dynamic obstacle avoidance methods offer precise avoidance capabilities and perform well in predictable environments. However, these methods are highly dependent on accurate environmental modeling. When the

model is inaccurate or the environment changes rapidly, frequent strategy adjustments are needed.

2.2 Model-free methods

Model-free obstacle avoidance algorithms avoid obstacles through real-time perception and local decision-making without relying on a detailed model or map of the environment. These methods use sensor data to respond dynamically to changes, process environmental information on the fly, and take immediate actions to prevent collisions.

Local path planning is an important model-free approach for dynamic obstacle avoidance. With this strategy, the robot continuously updates its understanding of the environment on the basis of current sensor input (such as LiDAR or cameras), allowing it to adapt to complex and rapidly changing conditions. Missura and Bennewitz (2019) proposed a dynamic collision model that effectively predicts potential collision risks using the traditional dynamic window approach (DWA). Li et al. (2024) introduced an improved artificial potential field (APF) for path planning and control, addressing the issues of goal inaccessibility and collision avoidance.

DRL obstacle avoidance methods learn avoidance strategies through the interaction between the agent and the environment and are also classic model-free approaches. Maoudj and Hentout (2020) proposed an efficient Q-Learning (EQL) algorithm for the planning mobile robot path, introducing enhanced state action representations, a refined reward function, and an optimized action selection strategy to significantly accelerate convergence and improve the quality of generated paths. Rusu et al. (2017) proposed a method that combines DRL with Sim-to-Real transfer. This approach enables models trained through DRL to be applied effectively in real-world navigation by overcoming the challenges faced by traditional reinforcement learning methods in practical applications, such as the differences between simulations and reality. Kiran et al. (2021) proposed a deep reinforcement learning framework that explicitly balances safety and efficiency in autonomous driving. By designing appropriate reward functions and introducing safety restrictions, they ensured that autonomous driving systems could make real-time and effective decisions in complex road environments. Cimurs et al. (2022) proposed a goal-driven exploration method using deep reinforcement learning for local navigation, enabling autonomous exploration without maps in unknown environments with improved performance over previous approaches. Xie and Dames (2023) developed a learning-based navigation strategy that combines lidar sequences, pedestrian motion data, and sub-goal signals to determine robot steering and velocity in crowded spaces. Using a velocity-obstacle-guided reward, the method achieved improved safety and efficiency in both simulations

and real-world trials, outperforming existing model-based and learning-based approaches.

3 Method

This paper presents a navigation framework based on deep reinforcement learning. The overall architecture is illustrated in Fig. 2. The short-term history of the radar data is transformed into a Cartesian coordinate system of size (80, 80), with obstacles represented by different numerical values across frames. This layer, along with the path waypoints, serves as the input and is processed separately. The perception enhancement module improved the spatial information of the radar data map. The enhanced features, together with the processed path point features, are then fed into the proximal policy optimization (PPO) network, which directly outputs the robot's next obstacle avoidance action.

3.1 Reinforcement learning

3.1.1 Problem formulation

The policy module is designed to learn an optimal navigation strategy through reinforcement learning. Although unimodal systems are generally less robust than multimodal approaches, they can still provide sufficient navigation performance in controlled, open outdoor environments. Therefore, in scenarios such as airports or open plazas, we propose the use of radar data to perceive the distribution of surrounding obstacles. The extracted features were then fed into a deep neural network. The enhanced LiDAR features are fused with the path waypoint features processed and subsequently passed to the policy network, which outputs the corresponding navigation policy π .

The complex nature of navigation decision making can be modeled as a partially observable Markov decision process (POMDP) (Boutilier 2002; Bellman 1957), which is defined by a six-tuple $[S, A, T, R, \Omega, O, \gamma]$. Here, S denotes the partially observable state space, A represents the action space, and T is the state transition model. R is the reward function, Ω denotes the observation space, O specifies the observation probability distribution, and $\gamma \in [0, 1]$ is the discount factor.

The operational mechanism of reinforcement learning can be described as follows: At a random time step t , a reinforcement learning agent receives an observation $o_t \in \Omega$ corresponding to the current partially observable state $s_t \in S$ and selects an action from the action space A . After executing the action, the agent receives a reward $r(s_t, a_t)$ and transitions to a new state $s_{t+1} \in S$ while also receiving a new observation $o_{t+1} \in \Omega$. The task was formulated as a finite-horizon problem with an episode duration of T steps. The

agent aims to learn an optimal control policy $\pi(a_t | o_t; \theta_\pi)$ that maximizes the expected cumulative return.

The core goal of DRL is to derive a policy that maximizes the expected long-term rewards π_θ , which is represented by a neural network, to maximize the expected discounted reward. This objective is typically formalized by optimizing the loss function $L(\theta)$.

$$L(\theta) = \max_{\pi_\theta} \mathbb{E} \left[\sum_{t=1}^{\infty} \gamma^{t-1} R(s_t, a_t) \right] \quad (1)$$

3.1.2 Reinforcement learning setup

- 1) Action space. In this study, a continuous action space is used to ensure smoother and more stable motion generation. The action space consists of two motion parameters, where V_x and W_z represent the linear and angular velocities, respectively. Taking into account the physical constraints of the hardware platform, the linear velocity V_x is limited to the range of $[0, 0.5]$ meters per second, and the angular velocity W_z is restricted to $[-2, 2]$ radians per second. To normalize the action space, we apply the scaling of the absolute maximum value by mapping the velocity range to the interval $[-1, 1]$.
- 2) Observation space. The observation space consists of two main components: historical radar frames and the most recent path points. All data within the observation space are represented in the local reference frame of the robot. Historical radar data include ten scans within a 0.5-second window, capturing the relative movement of obstacles with respect to the agent. This allows the agent to better predict and respond to pedestrian movement, thus providing a significant advantage during obstacle avoidance. The radar data was then transformed from the polar coordinates to Cartesian coordinates and an array of 80×80 grids was constructed using these data. In this grid, the pixel values corresponding to the obstacles were represented by their respective data orders. The size of the grid effectively balances spatial resolution and computational efficiency.
- 3) Reward function. The reward function serves to quantify the immediate return an agent receives upon executing an action within a given environment state. The purpose of the reward function is to guide the agent in learning an optimal policy through feedback, ultimately achieving long-term goals. The reward function includes both positive rewards and penalties, in order to encourage the agent to reach the target point more quickly and with a shorter path, while minimizing collisions with obstacles. To guide the agent toward these objectives, four different reward components were designed: forward

motion reward R_a^t , collision penalty R_c^t , angular velocity penalty R_r^t , and adaptive navigation heading reward R_θ^t .

First, when the agent reached the designated destination successfully, it received a significant positive reward. In constriction, if the agent did not reach the destination within the time limit, a substantial penalty was applied. Additionally, during navigation, the agent is rewarded as it approaches the target, whereas a penalty is imposed if it moves further away from the target. The forward motion reward R_a^t is defined as follows:

$$R_a^t = \begin{cases} R_{\text{goal}} & \text{if } \|p_g^t\| < L_m \\ -R_{\text{goal}} & \text{if } t \geq t_{\text{max}} \\ r_{\text{path}}(\|p_g^{t-1}\| - \|p_g^t\|) & \text{otherwise.} \end{cases} \quad (2)$$

where the symbol p_g^t represents the position of the target in the robot's local coordinate system. L_m was set to 0.3 meters. $R_{\text{goal}} = 20$, $r_{\text{path}} = 3.2$.

In addition, the agent must maintain a safe distance from obstacles. This study defines two types of proximity penalties: when the distance between the agent and the obstacle approaches the radius of the robot, it is considered a collision, and this distance is defined as the collision distance d_r , which incurs a significant penalty; when the distance between the robot and the obstacle is smaller than a predefined danger distance d_m , a smaller penalty is applied. The collision avoidance penalty R_c^t is defined as follows:

$$R_c^t = \begin{cases} R_{\text{col}} & \text{if } \|p_d^t\| \leq d_r \\ R_{\text{obs}}(d_m - \|p_d^t\|) & \text{else if } \|p_d^t\| \leq d_m \end{cases} \quad (3)$$

where $\|p_d^t\|$ denotes the distance to the obstacle in the robot's local coordinate system. $R_{\text{col}} = -20$, $d_r = 0.3m$, $d_m = 1.2m$, $R_{\text{obs}} = -0.2$.

To prevent instability in agent motion caused by sudden changes in angular velocity, this study proposes a dynamic threshold-based angular velocity penalty R_r^t . This mechanism implements smooth control over turning movements via a linear penalty function. The designed angular velocity penalty mechanism incorporates three key characteristics: progressive penalization, safety constraints, and real-time adjustments. Specifically, when the angular velocity w_z exceeded a predefined threshold, a penalty proportional to the overshoot was applied. In addition, a maximum penalty limit p_{max} is enforced to avoid excessive suppression of necessary steering actions. All penalty values were dynamically adjusted in real time according to the current angular velocity.

$$R_r^t = \begin{cases} \max(r \cdot (|w_z| - w_\theta), -p_{\text{max}}) & \text{if } w_\theta < w_z \\ 0 & \text{otherwise} \end{cases} \quad (4)$$

where the rotation penalty weight r is set to -0.2 , $w_\theta = 0.5$ rad/s is the angular velocity threshold, and $p_{max} = 0.5$ is the maximum penalty.

An adaptive navigation heading reward R_θ^i is introduced to address the directional adjustment during agent movement.

$$R_\theta^i = w_\theta \cdot (\theta_{max} - |\theta_{best} - \theta_g|) \quad (5)$$

where $w_\theta = 0.6$ denotes the reward weight, $\theta_{max} = \pi/6$ represents the threshold for angular deviation, θ_{best} denotes the feasible direction that most closely aligns with the direction of the sub-goal, and θ_w is the direction of the sub-goal.

The purpose of the adaptive heading reward is to guide the robot toward the target while avoiding collisions. It does so by selecting the optimal feasible direction until the goal is reached. As illustrated in Fig. 1, an obstacle cone is constructed between the robot and the surrounding obstacles. The detailed method for determining the optimal navigation angle is as follows.

Let the current position of the robot be at the origin $(0,0)$, and let the target position be (x_g, y_g) . The angle of target direction θ_g relative to the robot coordinate frame is defined as:

$$\theta_g = \text{clip}\left(\arctan 2(y_g, x_g), -\frac{\pi}{2}, \frac{\pi}{2}\right) \quad (6)$$

where the angle is restricted to the LiDAR field of view to maintain feasibility.

The LiDAR scan data are processed by subsampling, where every six points are examined, and the nearest one is retained as a representative obstacle point. Furthermore, to reduce the influence of distant noise and emphasize local obstacle avoidance, only observations within a 5 meter range are considered effective obstacle information. Consequently, obstacles P_b can be represented by LiDAR measurements (r_b^i, θ_b^i) that satisfy the above conditions, where i denotes the

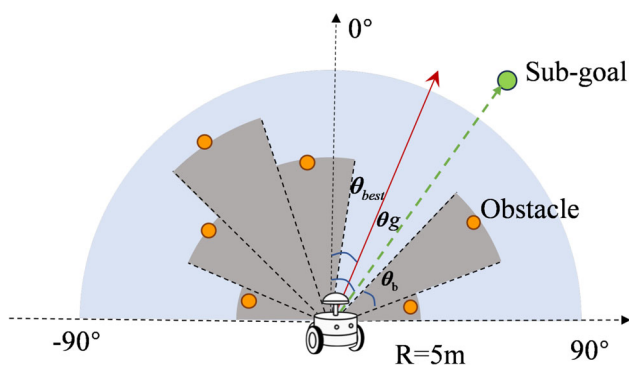


Fig. 1 Schematic illustration of the angular obstacle sectors (gray fan-shaped regions) and the optimal driving direction (red arrow)

Algorithm 1 Adaptive selection of navigation heading

θ_{best} .

Input : Sub-goal heading angle θ_g , extract obstacle $[r, \theta_b]$ positions by sampling every 6th LiDAR point, $0 < r < 5, \theta_b \in [-\pi/2, \pi/2]$

Output: Optimal direction angle θ_{best}

```

1 Initialize :  $\theta_{best} \leftarrow \pi/2$ 
  if  $P_b \neq 0$ , then
2    $\theta_{min} \leftarrow \infty$ ;
3   for  $I = 1, 2, \dots, N$  do
4      $\theta_s \leftarrow \text{sample from } [-\pi/2, \pi/2]$ 
5      $\text{free} \leftarrow \text{True}$ ;
6     for  $p$  in  $P_b$  do
7        $\theta_d = \arctan 2(\sin(\theta_s - \theta_b), \cos(\theta_s - \theta_b))$ 
8       if  $\theta_d \leq 0.2$  then
9          $\text{free} \leftarrow \text{False}$ 
10        break
11      end
12      if  $\text{free}$  then
13        if  $\|\theta_s - \theta_g\| \leq \theta_{min}$  then
14           $\theta_{min} \leftarrow \|\theta_s - \theta_g\|$ 
15           $\theta_{best} \leftarrow \theta_s$ 
16        end
17      end
18    end
19  end
20 else
21    $\theta_{best} \leftarrow \theta_g$ 
22 end
23 return  $\theta_{best}$ 

```

index of each obstacle.

$$P_b = (r_b^i, \theta_b^i) \quad (7)$$

Within the LiDAR field of view, a set of candidate heading angles θ_s is sampled. Any angle that is too close to the direction of the obstacle is considered infeasible and excluded from selection.

$$\theta_s = -\frac{\pi}{2} + i \cdot \frac{\pi}{N}, \quad i \in [0, N-1], N = 60 \quad (8)$$

The feasible forward direction θ_s must satisfy the obstacle avoidance constraint: for every obstacle direction θ_b , the angular separation from θ_s must be at least 0.2 rad, which is considered a safe margin for navigation.

$$\forall \theta_b, |\theta_s - \theta_b| \geq 0.2 \text{ rad}, \quad (9)$$

Among the feasible directions, the highest reward is assigned to the one closest to the sub-goal, guiding the agent to converge toward a direction that both approaches the target and avoids obstacles. For each feasible direction $\theta_s \in \Theta$, we compute its angular deviation from the sub-goal direction θ_g . The current minimum deviation θ_{min} is initialized to infinity to ensure that the first feasible direction is selected. If the

deviation is smaller than $\theta_{\min}$, we update $\theta_{\min}$ and set θ_{best} to θ_s . The update procedure is defined as follows:

$$\theta_{\min} = \arg \min_{\theta \in \Theta} (\text{dist}(\theta_{\text{best}}, \theta_g)^2) \quad (10)$$

where $\text{dist}(a, b) = \arctan 2(\sin(a-b), \cos(a-b))$ computes the angle difference wrapped.

During the reinforcement learning optimization process, the policy continuously adjusts its action selection to maximize reward, which is equivalent to minimizing $\theta_{\min}$. Therefore, in terms of convergence, the policy tends to align with the direction θ_g , the feasible heading that is closest to the target direction while meeting safety constraints.

The adaptive heading reward guides the robot to move as closely as possible toward the target while avoiding collisions by encouraging the selection of the optimal feasible heading direction. Its design ensures that the reward is continuous and bounded, facilitating convergence of reinforcement learning algorithms under standard assumptions of a Markov decision process. Furthermore, random sampling of candidate angles guarantees coverage of feasible directions, allowing the learned policy to converge to a robust navigation strategy.

3.1.3 Generalization analysis

In reinforcement learning, generalization refers to the ability of a policy to achieve high expected returns in environments not encountered during training, formalized as:

$$\mathbb{E}_{M \sim \mathcal{M}_{\text{test}}} [J(\pi, M)] \approx \mathbb{E}_{M \sim \mathcal{M}_{\text{train}}} [J(\pi, M)] \quad (11)$$

where π denotes the learned policy and M denotes a Markov Decision Process, where $J(\pi, M)$ represents the expected return in M . The difference in expected returns between training and testing environments can be quantified as:

$$\Delta J = |\mathbb{E}_{M \sim \mathcal{M}_{\text{test}}} [J(\pi, M)] - \mathbb{E}_{M \sim \mathcal{M}_{\text{train}}} [J(\pi, M)]| \quad (12)$$

Assuming the environment dynamics satisfy Lipschitz continuity, $\|P_{\text{test}}(s'|s, a) - P_{\text{train}}(s'|s, a)\|_1 \leq \epsilon$, $\forall s, a$, with $P(s'|s, a)$ denoting the state transition probability, RL theory yields an upper bound on expected return differences:

$$\Delta J \leq \frac{R_{\max}}{1 - \gamma} \epsilon \quad (13)$$

where ϵ quantifies the largest difference in state transition probabilities between the training and test environments, and $R_{\max}$ is the maximum reward while γ is the discount factor.

The proposed DRL navigation algorithm uses LiDAR point clouds and subgoal points as input. By retaining only invariant geometric information or relative positions and incorporating temporal aggregation with attention, the policy can be characterized via a function-space generalization bound.

$$\mathbb{E}_{M_{\text{test}}} [\ell(\pi(s), a)] \leq \mathbb{E}_{M_{\text{train}}} [\ell(\pi(s), a)] + \mathcal{C}(\mathcal{F}, n) \quad (14)$$

where ℓ is the loss function, $\mathcal{F}$ represents the policy function space, n is the number of training samples and $\mathcal{C}$ is the generalization bound, which depends on the complexity of the function space and the amount of training data. Consequently, the generalization can be expressed as follows:

$$\Delta J \leq f(\epsilon, R_{\max}, \gamma, \mathcal{F}, n) \quad (15)$$

Given these conditions, the finiteness of the generalization bound formally ensures that the learned policy exhibits reliable generalization from the training distribution to novel environments. Temporal aggregation of point clouds, combined with attention mechanisms, allows the model to focus on invariant spatial-temporal features instead of scene-specific layouts. This design supports effective generalization, enabling robust navigation in environments with diverse pedestrian densities and dynamics.

3.2 Overall network architecture

The proposed network follows an end-to-end architecture that maps temporal LiDAR data and sub-goal positions directly to continuous control commands. As shown in Fig. 2, the input to the network consists of a sequence of preprocessed radar frames and a subgoal vector, which together capture both spatial and task-specific information. The proposed module comprises four key components: shallow convolutional feature extraction, deep feature extraction, attention mechanism enhancement, and fully connected encoding.

Initially, the temporal LiDAR data are passed through two consecutive 3×3 convolutional layers with 32 and 64 channels to extract local spatial structural features and expand the receptive field, thus establishing a foundation for a deep understanding of the scene. The extracted intermediate features are then fed into an MBConvBlock module inspired by the EfficientNet architecture (Atila et al. 2021), which integrates separable convolutions in depth with lightweight residual connections to achieve a balance between performance and parameter efficiency. Multiple stacked MBConvBlock structures (each MBConvBlock is illustrated in Fig. 3(a)) within the module further enhance the network's ability to model semantic elements such as obstacles and traversable regions in complex environments.

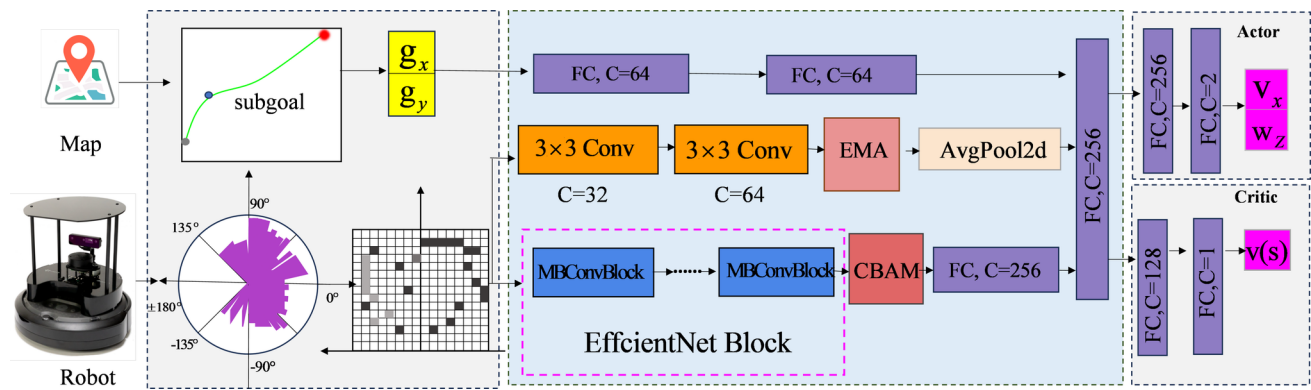


Fig. 2 Overall architecture of the proposed DRL-based navigation network. Data from the LiDAR and the target point are fed into the perception enhancement module. The resulting semantic features are then separately input into the actor and critic networks. The fully connected

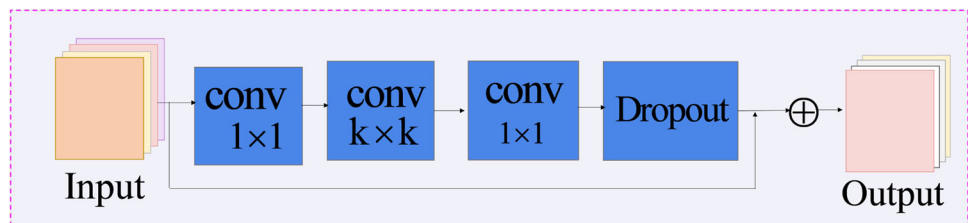
(FC) layer transforms the concatenated features into an N-dimensional output, with $C=N$ indicating the number of output channels used as the final representation for downstream tasks

The framework incorporates a convolutional block attention module (CBAM) (Woo et al. 2018) to enhance feature selection by highlighting important channels and spatial locations. Shallow features are further refined with an efficient multiscale attention module (EMA) (Ouyang et al. 2023) and compressed into a fixed-size global representation via

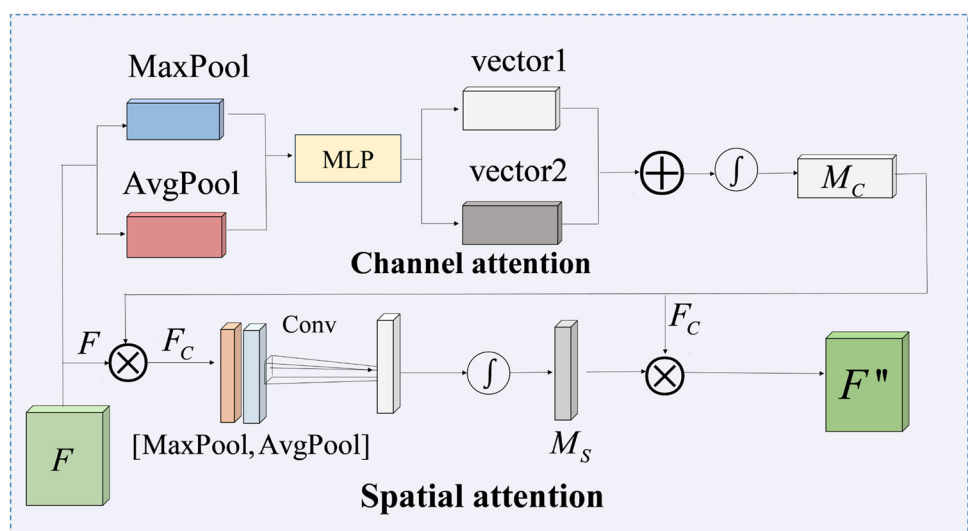
adaptive average pooling, improving temporal stability and semantic aggregation. Finally, a fully connected layer maps the features into a semantic embedding vector, which serves as input to the actor and critic networks.

The PPO network adopts an actor–critic architecture. The critic estimates the state value $v(s)$ by output a scalar through

Fig. 3 The structural of MBConvBlock and CBAM attention module



(a)



(b)

two fully connected layers. The actor generates continuous actions $[V_x, W_z]$, allowing real-time decision making in dynamic environments.

3.2.1 CBAM attention module

The CBAM attention module, which includes both channel and spatial attention components, is introduced to improve the feature selection capability of the model. These sub-modules dynamically recover feature maps by adjusting the importance of different channels and spatial locations, thereby highlighting information that is highly relevant to the navigation task.

The general architecture of the CBAM attention module is illustrated in Fig. 3(b). For an input map $F \in \mathbb{R}^{C \times H \times W}$, the channel attention map M_c is computed as

$$M_c(F) = \sigma(\text{MLP}(\text{AvgPool}(F)) + \text{MLP}(\text{MaxPool}(F))) \quad (16)$$

where AvgPool and MaxPool perform global aggregation in spatial dimensions, MLP is a shared multilayer perceptron, and σ denotes activation of the sigmoid. The channel-refined feature map is obtained by multiplication in element-wise with the original feature:

$$F_c = M_c(F) \otimes F \quad (17)$$

The spatial attention module generates a spatial attention map $M_s \in \mathbb{R}^{1 \times H \times W}$, from the channel-refined feature F_c :

$$M_s(F_c) = \sigma(f^{7 \times 7}([\text{AvgPool}_c(F_c); \text{MaxPool}_c(F_c)])) \quad (18)$$

where AvgPool_c and MaxPool_c form groups along the channel dimension, and $f^{7 \times 7}$ is a convolution with the kernel size 7×7 . The final output feature map is obtained by applying the spatial attention map to the channel-refined feature:

$$F'' = M_s(F_c) \otimes F_c \quad (19)$$

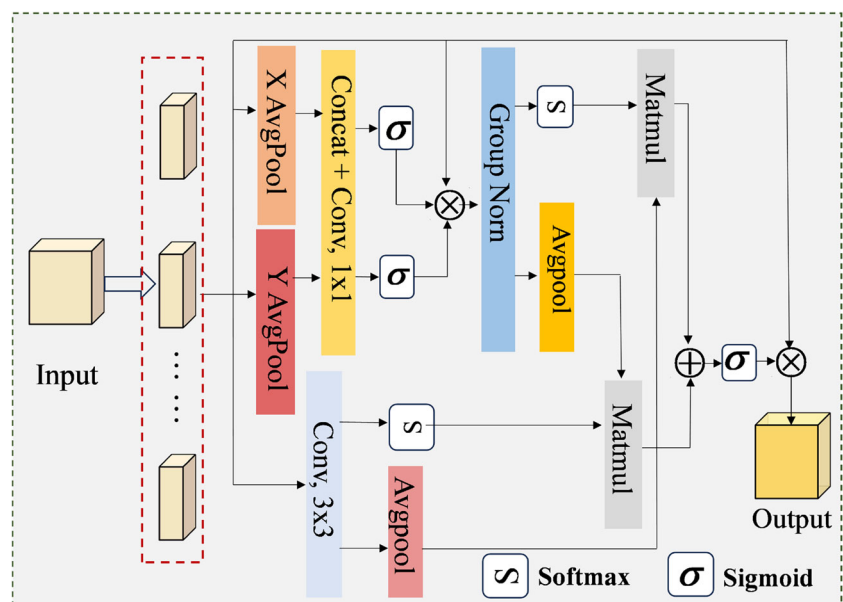
By independently generating channel and spatial attention maps and sequentially applying them to the original feature map, CBAM effectively enhances discriminative feature representation with minimal computational overhead.

3.2.2 EMA attention module

The EMA module aims to improve the representational quality of convolutional neural networks by capturing long-range dependencies and efficiently aggregating multiscale contextual information. The module generates an adaptive attention map that selectively emphasizes informative features while suppressing less relevant ones.

As shown in Fig. 4, given a feature map $F \in \mathbb{R}^{C \times H \times W}$, EMA partitions it into subgroups G , where each group is responsible for learning different semantic representations. This grouping not only facilitates parallel processing across multiple GPUs, but also enhances semantic feature learning while suppressing noise. EMA employs three parallel branches to extract attention descriptors. Two branches use 1×1 convolutions with one-dimensional global average pooling to encode channel information along horizontal and vertical directions, while the third branch applies a 3×3 convolution to capture multiscale features. This design allows EMA to model inter-channel dependencies and preserve spatial structure information simultaneously.

Fig. 4 The structural of the EMA attention mechanism



At the spatial level, EMA enriches feature aggregation through cross-dimensional fusion. Specifically, the 1×1 branch output undergoes a two-dimensional global average pooling to encode global spatial context, while the 3×3 branch output retains its spatial structure. These outputs are fused via the matrix dot product to form the initial spatial attention map. Finally, each group's feature maps are weighted by two Sigmoid-normalized attention values, emphasizing contextual relationships at the pixel level. The output of EMA has the same size as the input, making it easily integrable into modern architectures.

4 Experimental evaluation

This study includes comprehensive evaluations conducted in both simulated and real-world environments. The experimental setup was carefully designed to rigorously assess the performance of the method under diverse operating conditions and benchmark it against the baseline algorithms. This section provides a systematic description of the configuration of the experimental platform, the policy training procedures, and the results of the performance evaluation.

4.1 Experimental setup

A hybrid validation approach was used to evaluate the performance of the proposed control strategy. Initially, a high-fidelity 3D testing environment was constructed using the Gazebo simulator (Fig. 5(a)), which provides realistic physics and environmental modeling. In parallel, pedestrian dynamics were modeled through the PEDSIM library (Camara et al. 2020) (Fig. 5(b)), allowing simulation of diverse and realistic crowd behaviors. For simulation experiments, a Turtlebot2 platform equipped with a Hokuyo UTM-30LX LiDAR was used, with a maximum speed limit of 0.5 m/s. The Hokuyo sensor was configured with a measurement range of [0.1, 30] meters, a field of view of 270°

and an angular resolution of 0.25° . PEDSIM is a microscopic pedestrian simulation library based on the Social Force Model, which reproduces realistic pedestrian dynamics by modeling multilevel mechanical interactions and achieving a high-fidelity behavioral simulation.

Figure 6 presents five simulated experimental maps. In each map, red dots represent target points, green areas denote starting locations, and black lines connecting the red dots indicate the navigation paths. Each map contains 25 short paths. Figure 6(a) shows the Lobby world (25×10 m), an open space with chairs, tables, and pillars, which serves as the primary training environment. Figure 6(b) depicts the Autolab world (20×10 m), featuring a multi-room layout to evaluate generalization in complex settings. The Cumberland world, shown in Fig. 6(c) (20×10 m), comprises extended corridors designed to test navigation in elongated layouts. Figure 6(d) illustrates the Freiburg world (20×10 m), a reconstruction of the architectural layout of the Freiburg University with corridors, rooms and intersections, used to assess performance in realistic academic buildings. Finally, Fig. 6(e) shows the Square world (20×20 m), an open environment with regularly shaped obstacles to examine navigation in artificially structured spaces. The model was trained in a lobby environment containing 34 pedestrians, where the robot repeatedly navigates between random start and target points to improve the robustness of the strategy. Subsequently, Its generalization was tested in five additional environments: Lobby (5 to 55 pedestrians), Autolab, Cumberland, Freiburg, and Square worlds (25 to 35 pedestrians each) to examine performance in different crowd densities and previously unseen scenarios.

This study adopted a multisenario, staged validation protocol to rigorously evaluate the performance and generalization capabilities of the DRL-based control strategy. The initial training phase was conducted in a lobby environment populated by 34 pedestrians. The strategy was subsequently evaluated in the same environment in varying crowd densities, with a number of pedestrians ranging from five to 55. Finally, the strategy was deployed in four distinct

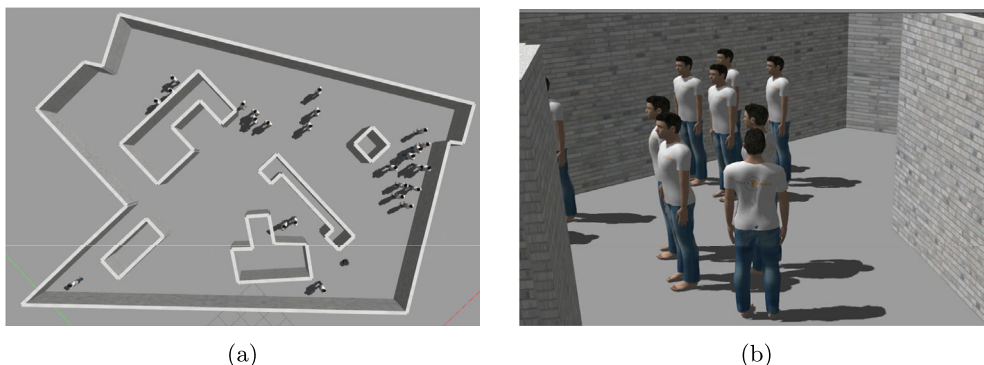


Fig. 5 The dense pedestrian simulation in a high-fidelity 3D environment

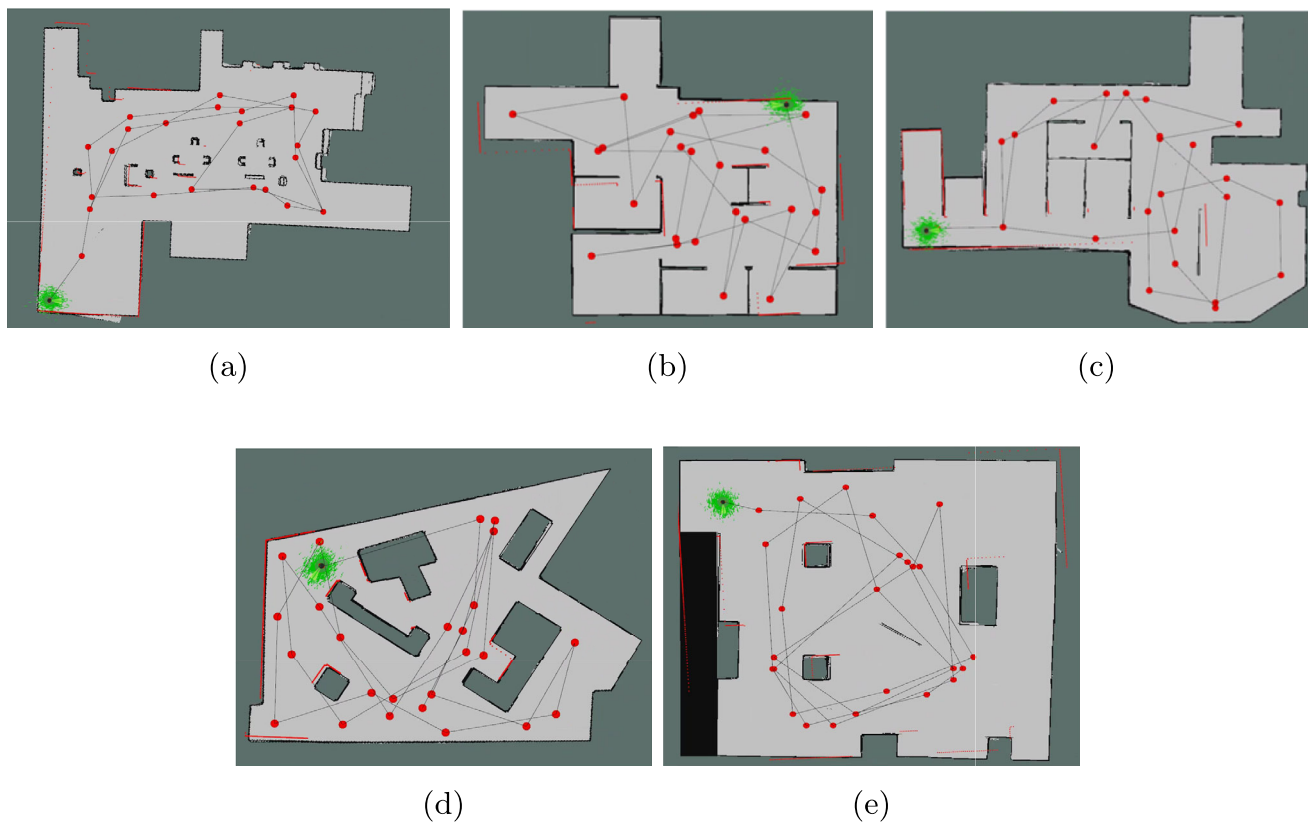


Fig. 6 The simulation maps for training. The training scenarios include the Lobby world with 34 pedestrians, as well as the Lobby, Autolab, Cumberland, Freiburg, and Square worlds with varying pedestrian densities ranging from five to 55 in increments of 10

environments to assess its generalizability in previously unseen contexts. Training and simulation experiments were conducted on a workstation equipped with a CPU with 20 threads and an NVIDIA GeForce RTX 3060 GPU with 12 GB GDDR6 memory. The system also runs Ubuntu 20.04 LTS with kernel version 5.15.b0-125-generic, and development and testing were performed within the ROS Noetic Ninjemys framework, utilizing the Gazebo 11.5.1 simulation environment.

4.2 Simulation results

4.2.1 Comparative experiments

We developed a navigation strategy based on the PPO algorithm (Yu et al. 2022; Long et al. 2018; Fan et al. 2020) to train the deep reinforcement learning control module. The hyperparameters used for the training model are summarized in Table 1. The initial learning rate was established at 10^{-3} , with a batch size of 512 and a step decay schedule applied during training. To examine the impact of different learning rates on training performance, Fig. 7 presents the reward trends over 1000 episodes in two settings: the learning rates of 3×10^{-4} and 10^{-3} , respectively. The horizontal axis

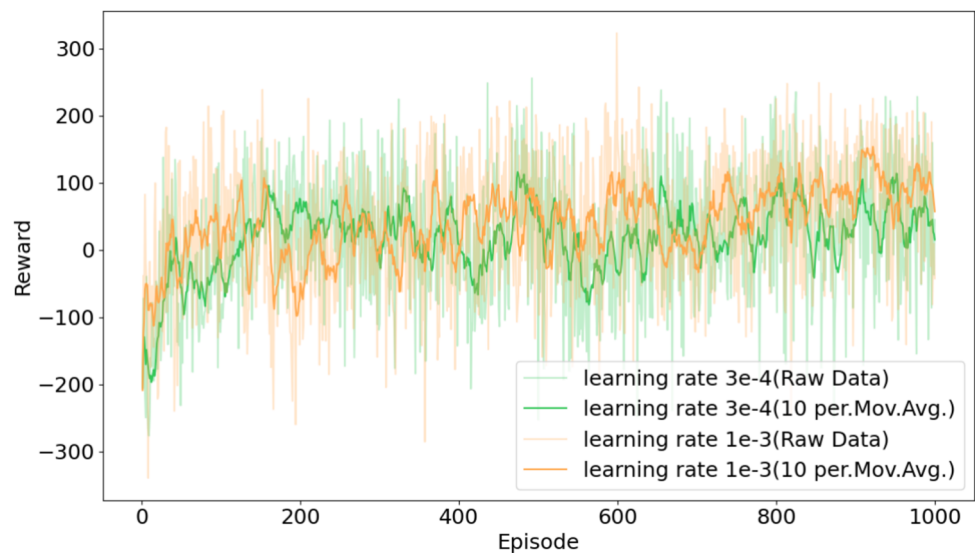
denotes the episode number, while the vertical axis shows the corresponding reward values. Each curve represents the moving average of the rewards, computed over the preceding 10 episodes. The reward curves for both learning rates show similar convergence, suggesting that the final performance of the model is stable with respect to this hyperparameter.

We perform a comparative evaluation of navigation performance in five representative environments: Lobby, Autolab, Cumberland, Square, and Freiburg. The baseline algorithms include unimodal methods based exclusively on LiDAR

Table 1 The configurations and hyperparameters for the proposed algorithm

Description	Parameter	Value
Discount factor	gamma	0.99
PPO learning rate	learning_rate	1e-3
Rollout step	n_steps	512
Training epochs per rollout	n_epochs	10
Batch size	batch_size	128
Total training timesteps	total_timesteps	512,000
Entropy coefficient	ent_coef	0
Value function coefficient	vf_coef	0.5

Fig. 7 The sensitivity of training reward to learning rate



perception, as well as multimodal methods that combine multiple sensing modalities. In particular, Modal baselines include reinforcement learning–based methods such as A1-RD (Guldenring et al. 2020), A1-RC (Guldenring et al. 2020), and GA-DRL (Cimurs et al. 2022), as well as classical planning methods, including DWA (Fox et al. 1997), MPC (Rösmann et al. 2021) and PSO (Kennedy and Eberhart 1995). In contrast, multimodal baselines comprise DRL (Xie and Dames 2023) and DRL-VO (Xie and Dames 2023), which use multiple sensing modalities for perception and decision making. For fair comparison, all baseline methods were implemented according to their original settings as described in the literature, without retraining. We introduced four commonly used metrics to assess performance in navigation tasks: success rate, average time, average length, and average speed. We conducted four rounds of tests via a consistent experimental setup and control strategy. In each round, the robot was required to sequentially reach 25 predefined navigation target points within the test environment, and all tests were carried out under identical initial conditions.

Table 2 summarizes the experimental results for each algorithm under varying crowd densities. The data suggest that our proposed method generally achieves high navigation performance and stability across a wide range of densities, from five to 55 people. In particular, it maintains relatively high navigation speeds with limited fluctuations in success rate, reflecting both efficiency and robustness. In comparison, classical planners tend to exhibit certain limitations. For example, DWA, which relies primarily on local sensor information, often produces shorter trajectories and shows occasional speed variability in dynamic environments. MPC demonstrates moderate stability in moderately complex scenarios, but its efficiency may decrease under high-density

conditions due to a restricted optimization horizon. PSO, which depends on global optimization objectives, can face challenges in adapting to rapidly changing crowded environments.

Learning-based approaches, including our DRL method and DRL-VO, benefit from accumulated training experience, allowing them to maintain robust navigation in high-density conditions by effectively balancing obstacle avoidance and speed control. A closer examination indicates that algorithm performance is strongly influenced by environmental characteristics. Methods such as PSO, GA-DRL, and A1-RD/RC perform reasonably well in low density or simple scenarios, while MPC and DWA are better suited for environments of moderate density and complexity. Our method, together with DRL-VO, consistently provides stable and efficient navigation in densely populated or highly dynamic settings, demonstrating strong adaptability in diverse conditions.

As shown in Table 3, comparative navigation experiments were conducted with 35 pedestrians in environments that were structurally distinct and had not been previously seen. These experiments were designed to assess the adaptability of each algorithm. Our algorithm generally showed strong performance, maintaining a high success rate while effectively balancing navigation speed and path efficiency. Slight performance degradation was observed in more complex environments, such as Freiburg World and Autolab World, which contain multiple rooms, narrow corridors, and frequent dynamic obstacles. In these scenarios, the algorithm occasionally selects suboptimal paths or pauses briefly, increasing path lengths and navigation times. This performance is influenced by several factors. The policy network tends to prioritize short-term local decisions, action constraints in confined spaces can cause brief pauses or speed reductions, and limited exposure to highly complex environments

Table 2 The navigation performance comparison with different pedestrian densities in Lobby World, the best results are highlighted in bold

Environment	Method	Success Rate ↑	Average Time (s) ↓	Average Length (m) ↓	Average Speed (m/s) ↑
Lobby world, 5 pedestrians	DWA	0.90	13.8	5.08	0.37
	MPC	0.92	22.06	5.95	0.27
	PSO	0.79	19.59	7.77	0.40
	A1-RD	0.92	19.9	8.43	0.42
	A1-RC	0.83	18.8	7.91	0.42
	DRL	0.90	13.2	5.98	0.45
	GA-DRL	0.62	21.17	8.68	0.41
	DRL-VO	0.93	12.38	5.22	0.42
	Ours	0.94	14.1	5.85	0.41
Lobby world, 15 pedestrians	DWA	0.92	14.2	7.94	0.56
	MPC	0.92	17.94	6.02	0.34
	PSO	0.76	20.81	7.57	0.36
	A1-RD	0.91	20.7	8.70	0.42
	A1-RC	0.85	19.87	8.75	0.44
	DRL	0.89	13.38	6.08	0.45
	GA-DRL	0.47	26.42	9.45	0.36
	DRL-VO	0.94	12.40	5.24	0.42
	Ours	0.94	14.4	5.80	0.40
Lobby world, 25 pedestrians	DWA	0.88	14.4	5.08	0.35
	MPC	0.87	18.32	6.04	0.33
	PSO	0.74	17.54	6.79	0.39
	A1-RD	0.87	24.6	7.38	0.30
	A1-RC	0.84	16.18	7.23	0.44
	DRL	0.87	13.04	5.98	0.46
	GA-DRL	0.28	34.84	9.98	0.29
	DRL-VO	0.93	12.50	5.27	0.42
	Ours	0.90	13.96	5.80	0.41
Lobby world, 35 pedestrians	DWA	0.80	15.81	5.13	0.32
	MPC	0.82	21.19	6.27	0.30
	PSO	0.70	17.51	6.46	0.37
	A1-RD	0.76	20.5	6.73	0.33
	A1-RC	0.72	19.08	8.10	0.42
	DRL	0.77	13.62	6.05	0.44
	GA-DRL	0.31	28.16	10.42	0.37
	DRL-VO	0.88	13.18	5.35	0.40
	Ours	0.90	14.5	6.02	0.41
Lobby world, 45 pedestrians	DWA	0.74	17.8	5.17	0.29
	MPC	0.73	20.27	6.16	0.305
	PSO	0.65	18.13	6.45	0.35
	A1-RD	0.74	44.3	9.42	0.21
	A1-RC	0.60	34.3	10.54	0.31
	DRL	0.65	14.97	6.49	0.43
	GA-DRL	0.32	29.81	11.88	0.40
	DRL-VO	0.78	14.91	5.30	0.36
	Ours	0.85	14.8	5.97	0.40

Table 2 continued

Environment	Method	Success Rate $\uparrow$	Average Time (s) $\downarrow$	Average Length (m) $\downarrow$	Average Speed (m/s) $\uparrow$
Lobby world, 55 pedestrians	DWA	0.72	21.8	5.22	0.24
	MPC	0.70	24.87	6.51	0.26
	PSO	0.55	23.07	7.55	0.26
	A1-RD	0.50	37.61	9.65	0.27
	A1-RC	0.54	33.28	11.24	0.34
	DRL	0.67	17.08	5.95	0.35
	GA-DRL	0.22	29.46	11.41	0.39
	DRL-VO	0.79	14.44	5.28	0.36
	Ours	0.78	14.10	5.75	0.41

during training further limits generalization. In contrast, the A1-RD policy exhibited strong performance in open environments, such as Cumberland and Square, primarily due to its obstacle avoidance mechanism, which is based on passive distance perception. Specifically, when an obstacle is detected ahead, the system guides the robot to turn toward open areas and transition to a waiting state. However, this static obstacle avoidance strategy, which is central to the training process, is insufficient in highly dynamic and densely populated environments, such as Autolab and Freiburg, leading to a significant reduction in navigation success rates. The GA-DRL algorithm demonstrates relatively limited obstacle avoidance capabilities, particularly in more complex environments such as the Cumberland and Freiburg, where it may occasionally have difficulty navigating around pedestrians. It can also sometimes exhibit redundant rotations or minor stalling near walls or narrow corners.

Figure 8 illustrates the long-range navigation trajectories of different algorithms in four experimental environments. The green circles denote the starting points, while the red pentagrams indicate the target locations. The black regions represent static obstacles, and the black dots with arrows denote dynamic pedestrians and their movement directions. Each environment includes 25 pedestrians. Successful navigation is defined as the robot reaching the designated target location. Figure 8(a) presents the navigation results in Cumberland World, where the robot starts at (0, 0) and the target is located at (22, 9). The A1-RC, DRL, and DRL-VO algorithms entered the rooms at corners and engaged in a prolonged rotational exploration, resulting in task failure. The A1-RD and GA-DRL algorithms exhibited the longest trajectories, primarily due to their limited obstacle avoidance capabilities, which often led to detours or inefficient exploration in complex environments. In contrast, the remaining algorithms successfully reached the target via relatively short paths. Figure 8(b) shows the navigation results in Autolab World, with the robot starting at (1, 0) and the target at (14, 10). Most of the algorithms successfully reached the goal,

while the GA-DRL algorithm continued to rotate within the room, failing to complete the task.

Figure 8(c) shows the navigation results in the Lobby World, where the robot starts at (1, 0) and the target is located among tables and chairs at (10.5, 13.5). The GA-DRL algorithm altered its original path to avoid dynamic obstacles and successfully reached the goal. Other algorithms initially attempted to reach the target via the right side of the tables but were unable to achieve the required turning angles, and thus had to adjust their routes. After changing direction, the DRL algorithm engaged in prolonged redundant rotations to avoid the tables, resulting in a task failure. The A1-RD algorithm was continuously explored along the periphery but failed to reach the target. Figure 8(e) presents the statistics of the distances between the robot and the nearest obstacles during the navigation task Fig. 8(c), with distances below 0.2 considered collisions. It is evident that the A1-RD, DWA and MPC algorithms experienced collisions, indicating limited reliability and safety.

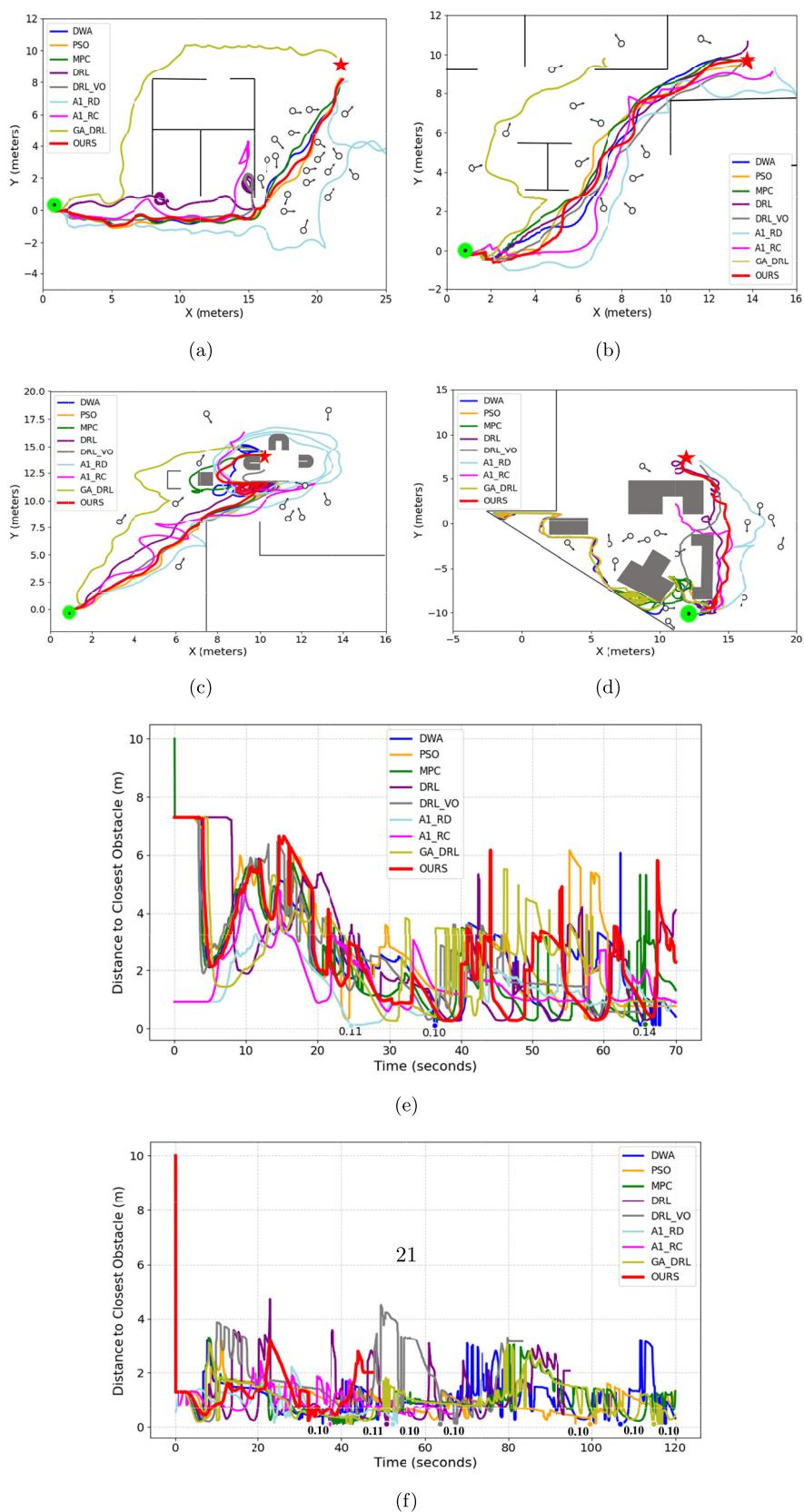
Figure 8(d) depicts the navigation results in Freiburg World, where the robot starts at (13.5, -10.5) and the target is at (12.5, 7). The DWA, GA-DRL, MPC, and PSO algorithms chose narrow corridors and ultimately became trapped in corners. Although the A1-RC algorithm selected the optimal route, it remained stuck while exploring the confined space. The DRL algorithm exhibited redundant unnecessary rotations during movement, increasing exploration costs. Figure 8(f) shows the distance statistics between the robot and the nearest obstacles for the navigation task in Fig. 8(d). Except for our algorithm and MPC, all other methods experienced collisions during navigation.

The experimental results show that the proposed algorithm achieves robust and efficient navigation in complex and densely populated environments. Traditional planning methods such as DWA, MPC, and PSO perform well in simple environments, but often fail in constrained or cluttered environments, colliding with obstacles, or getting stuck in local loops. DRL-based methods also show good

Table 3 The navigation performance comparison in different environments with 35 pedestrians, the best results are highlighted in bold

Environment	Method	Success Rate $\uparrow$	Average Time (s) $\downarrow$	Average Length (m) $\downarrow$	Average Speed (m/s) $\uparrow$
Lobby world, 35 pedestrians	DWA	0.80	15.81	5.13	0.32
	MPC	0.82	21.19	6.27	0.30
	PSO	0.70	17.51	6.46	0.37
	A1-RD	0.84	20.5	6.73	0.33
	A1-RC	0.72	19.08	8.10	0.42
	DRL	0.77	13.62	6.05	0.44
	GA-DRL	0.31	28.16	10.42	0.37
	DRL-VO	0.88	13.18	5.35	0.40
	Ours	0.90	14.5	6.02	0.41
Autolab world, 35 pedestrians	DWA	0.76	22.30	5.58	0.25
	MPC	0.79	25.15	7.16	0.29
	PSO	0.55	28.24	8.56	0.30
	A1-RD	0.75	23.22	7.42	0.31
	A1-RC	0.60	36.96	12.09	0.32
	DRL	0.67	17.98	7.66	0.42
	GA-DRL	0.28	22.74	8.27	0.37
	DRL-VO	0.83	15.8	6.54	0.41
	Ours	0.84	19.94	6.72	0.34
Cumberland world, 35 pedestrians	DWA	0.70	20.50	5.60	0.27
	MPC	0.63	32.84	7.06	0.21
	PSO	0.60	27.48	8.23	0.3
	A1-RD	0.78	14.80	6.60	0.44
	A1-RC	0.54	35.04	12.50	0.36
	DRL	0.68	19.16	7.74	0.40
	GA-DRL	—	—	—	—
	DRL-VO	0.78	14.5	5.96	0.41
	Ours	0.80	17.15	6.77	0.39
Freiburg world, 35 pedestrians	DWA	0.72	22.10	5.90	0.28
	MPC	0.64	31.76	7.66	0.24
	PSO	0.50	22.95	7.78	0.34
	A1-RD	0.72	22.24	7.97	0.36
	A1-RC	0.44	36.43	11.78	0.32
	DRL	0.64	29.10	11.30	0.39
	GA-DRL	—	—	—	—
	DRL-VO	0.76	16.1	6.5	0.40
	Ours	0.74	18.66	7.21	0.38
Square world, 35 pedestrians	DWA	0.88	21.53	8.40	0.39
	MPC	0.91	30.47	10.13	0.33
	PSO	0.87	21.13	9.64	0.46
	A1-RD	0.96	21.98	11.76	0.54
	A1-RC	0.72	30.20	14.22	0.37
	DRL	0.82	22.40	10.00	0.44
	GA-DRL	0.32	28.52	10.07	0.35
	DRL-VO	0.86	19.3	8.61	0.44
	Ours	0.90	22.08	9.60	0.43

Fig. 8 The comparison of navigation trajectories and robot-obstacle minimum distance over time for various algorithms in 25-person environments



performance in simple scenarios. However, their navigation ability degrades significantly in complex environments due to limited dynamic obstacle avoidance. GA-DRL can navigate without maps, but is prone to termination after collisions with walls and is highly susceptible to dynamic obstacles. A1-RD reduces collision risk through a stop-and-wait mechanism but sacrifices navigation speed in crowded environments, while A1-RC tends to take detours in complex scenes, leading to longer travel distances. DRL-VO demonstrates strong performance through the integration of multisensor fusion and pedestrian motion prediction. Our method improves adaptability by integrating spatio-temporal obstacle information with an adaptive heading reward to mitigate the passivity and waiting of traditional methods.

4.2.2 Extended experiments

In this set of extended experiments, we evaluate various aspects of the proposed navigation system, including real-time performance, the impact of random seeds on training, the effect of the LiDAR frame, rate, and the role of the EMA attention mechanism. These studies aim to provide a comprehensive assessment of the efficiency, robustness, and capability of the feature representation of the method.

A systematic evaluation of six deep reinforcement learning-based navigation algorithms was conducted, with particular attention paid to their model size, inference speed, and GPU memory consumption. As shown in Table 4, lightweight models such as A1-RD and A1-RC have minimal parameters and achieve very high frame rates, but their limited capacity may restrict performance in dense or dynamic environments. Midsized models like GA-DRL offer a trade-off between efficiency and feature expressiveness, whereas more complex models such as DRL and DRL-VO contain significantly more parameters and run at lower FPS. Our proposed model has the largest parameter count and a relatively lower inference speed, yet this design allows it to leverage richer multi-modal features and attention mechanisms, which are critical for complex navigation tasks. In general, these results highlight that our approach prioritizes representational power over raw speed, reflecting a deliberate balance between computational cost and navigation performance.

Table 5 summarizes the navigation performance in four different random seeds. In all environments, the model achieves an average success rate ranging from 0.72 to 0.85, with relatively small standard deviations, indicating that the choice of random seed has a modest impact on success rates. In contrast, metrics such as average speed and navigation time exhibit larger variations between seeds, reflecting greater sensitivity to stochastic factors. Overall, these results demonstrate that our algorithm maintains robust navigation performance, while certain temporal metrics remain affected by inherent randomness.

To examine the impact of temporal information on navigation performance, we conducted experiments using one, five, and ten historical frames. As shown in Table 6, single-frame input consistently led to navigation failures, with the vehicle repeatedly turning in place. Using multiple frames (five or ten) significantly improved navigation success rates by about 12.5 percentage points, although travel efficiency decreased slightly. In particular, using more LiDAR frames helps capture the spatio-temporal features of dynamic obstacles, further improving navigation performance and reducing collisions. These results indicate that the incorporation of temporal information is critical for robust navigation in dynamic environments.

The weights computed by the EMA attention mechanism are visualized as heat maps in Fig. 9. The visualization shows that the model assigns positive weights to unoccluded regions and negative weights to obstacle regions, indicating that the EMA attention mechanism effectively distinguishes traversable space from obstacles. As the number of obstacles increases, the positive weights in open areas decrease, reflecting the redistribution of attention among more obstacles. These results suggest that the EMA attention mechanism dynamically adjusts the weighting of features based on the complexity of the scene, supporting robust perception and decision making in cluttered environments.

4.2.3 Ablation study

To investigate the impact of different environmental settings on pedestrian simulation, we conducted ablation studies in two scenarios: the Lobby world and the Freiburg world.

Table 4 The comparison of model size, inference efficiency, and GPU resource usage across different algorithms

Method	Number of Parameters	FPS	GPU Memory Footprint (MB)
A1-RD	1.6 MB	1404.9	24 MB
A1-RC	1.6 MB	1550.2	24 MB
DRL	32.1 MB	346.5	214 MB
DRL-VO	32.1 MB	335.6	214 MB
GA-DRL	6.0 MB	1937.3	146 MB
Ours	57.8 MB	137.1	230 MB

Table 5 The effect of random seed initialization on model training performance

Environment	Random seed value	Success Rate $\uparrow$	Average Time (s) $\downarrow$	Average Length (m) $\downarrow$	Average Speed (m/s) $\uparrow$
Lobby world, 35 pedestrians	10	0.83	13.33	5.57	0.42
	3407	0.88	11.82	5.62	0.47
	1337	0.84	15.02	5.78	0.38
	2023	0.84	21.58	5.07	0.23
	Mean (SD)	0.85 $\pm$ 0.02	15.44 $\pm$ 4.30	5.51 $\pm$ 0.31	0.38 $\pm$ 0.10
Freiburg world, 35 pedestrians	10	0.70	16.46	6.43	0.39
	3407	0.76	17.16	7.73	0.45
	1337	0.70	17.22	6.12	0.36
	2023	0.71	29.45	6.20	0.21
	Mean (SD)	0.72 $\pm$ 0.03	20.07 $\pm$ 6.26	6.62 $\pm$ 0.75	0.35 $\pm$ 0.10
Lobby world, 45 pedestrians	10	0.76	13.50	5.47	0.40
	3407	0.84	11.6	5.45	0.47
	1337	0.81	15.09	5.67	0.37
	2023	0.78	28.79	6.32	0.21
	Mean (SD)	0.80 $\pm$ 0.03	17.24 $\pm$ 7.83	5.73 $\pm$ 0.41	0.36 $\pm$ 0.11
Lobby world, 55 pedestrians	10	0.72	13.16	5.50	0.42
	3407	0.83	11.74	5.49	0.47
	1337	0.78	16.08	6.03	0.37
	2023	0.76	28.58	6.22	0.22
	Mean (SD)	0.77 $\pm$ 0.05	17.39 $\pm$ 7.68	5.81 $\pm$ 0.37	0.37 $\pm$ 0.11

In addition, three high-density pedestrian versions were retained in the Lobby world to further evaluate the effectiveness of the modules in more extreme and crowded environments. We constructed two ablation settings: (1) progressively removing the CBAM and EMA attention modules while retaining the complete reward functions, and

(2) progressively removing four different reward functions while keeping the attention modules. Key evaluation metrics include success rate, average navigation time, path length, and average speed.

As shown in Table 7, w/o denotes the removal of a module, and the boldface values indicate the best results. The results

Table 6 The effect of temporal LiDAR frame aggregation on navigation performance

Environment	LiDAR frames	Success Rate $\uparrow$	Average Time (s) $\downarrow$	Average Length (m) $\downarrow$	Average Speed (m/s) $\uparrow$
Lobby world, 35 pedestrians	A single frame	—	—	—	—
	Five frames	0.81	13.23	5.58	0.42
	Ten frames (Ours)	0.90	14.50	6.02	0.41
Freiburg world, 35 pedestrians	A single frame	—	—	—	—
	Five frames	0.6	20.40	6.73	0.33
	Ten frames (Ours)	0.74	18.66	7.21	0.38
Lobby world, 45 pedestrians	A single frame	—	—	—	—
	Five frames	0.72	14.62	5.67	0.39
	Ten frames (Ours)	0.85	14.8	5.97	0.40
Lobby world, 55 pedestrians	A single frame	—	—	—	—
	Five frames	0.64	14.31	5.66	0.40
	Ten frames (Ours)	0.78	14.10	5.75	0.41

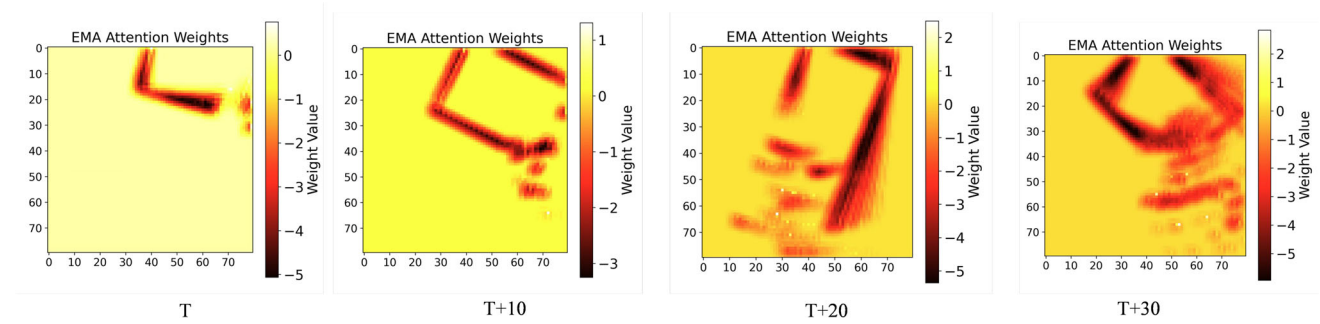


Fig. 9 The attention maps generated by the EMA attention module

demonstrate that the proposed method consistently outperforms attention-ablated baselines in different scenarios and pedestrian densities. For instance, in the Lobby world with 35 pedestrians, the full model achieves a notable improvement in success rate while reducing both path length and navigation time; similar advantages are observed in the Freiburg world, confirming the robustness of the method in complex environments. Although overall success rates decrease as pedestrian density increases, the full model maintains a clear advantage, indicating that the dual-attention mechanism (CBAM and EMA) effectively cooperates to enhance navigation performance in high-density dynamic crowds.

As shown in Table 8, it is evident that each module contributes significantly to navigation performance. In the Lobby world with 35 pedestrians, removing NavHR (adaptive navigation heading reward) results in a lower success rate, demonstrating that it is crucial to effectively steering the agent toward the target. Removing CAP (collision avoidance penalty) slightly reduces the success rate while decreasing the

average path length, indicating that the absence of collision penalties may shorten paths at the expense of safety. Removing AVP (angular velocity penalty) substantially affects both success rate and speed, demonstrating its importance for maintaining smooth and efficient movement. When the FMR (forward motion reward) is removed, the agent remains largely stationary, failing to navigate effectively. In the Freiburg world, navigation difficulty is higher, and overall success rates are lower than in the Lobby world; however, the complete model still achieves the highest success rate as pedestrian density increases to 45 and 55, confirming the effectiveness of multi-module cooperation in dense, dynamic crowds. In summary, NavHR improves goal-directed behavior, CAP ensures safe collision avoidance, AVP maintains smooth motion, and FMR facilitates efficient forward movement: removal of any module leads to degraded navigation performance.

The reward functions generated during the training process are analyzed in the ablation experiments. Specifically,

Table 7 The ablation analysis of the proposed algorithm on navigation performance in various environments

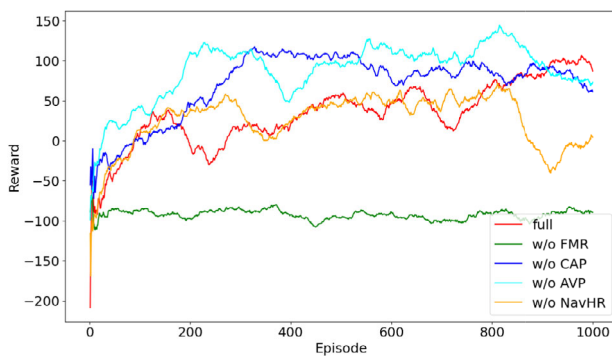
Environment	method	Success Rate $\uparrow$	Average Time (s) $\downarrow$	Average Length (m) $\downarrow$	Average Speed (m/s) $\uparrow$
Lobby world, 35 pedestrians	w/o CBAM attn	0.82	14.68	6.14	0.42
	w/o EMA attn	0.84	15.20	6.20	0.40
	Ours	0.90	14.50	6.02	0.41
Freiburg world, 35 pedestrians	w/o CBAM attn	0.69	18.42	7.47	0.40
	w/o EMA attn	0.68	19.92	7.33	0.37
	Ours	0.74	18.66	7.21	0.38
Lobby world, 45 pedestrians	w/o CBAM attn	0.78	14.94	6.17	0.41
	w/o EMA attn	0.80	15.7	6.27	0.40
	Ours	0.85	14.8	5.97	0.40
Lobby world, 55 pedestrians	w/o CBAM attn	0.74	14.22	5.88	0.41
	w/o EMA attn	0.73	14.35	5.72	0.40
	Ours	0.78	14.10	5.75	0.41

Table 8 The ablation study on navigation performance with respect to key reward and penalty modules

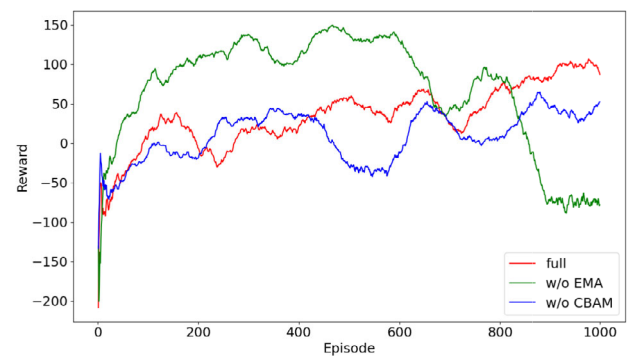
Environment	method	Success Rate $\uparrow$	Average Time (s) $\downarrow$	Average Length (m) $\downarrow$	Average Speed (m/s) $\uparrow$
Lobby world, 35 pedestrians	w/o NavHR	0.76	15.07	6.27	0.40
	w/o FMR	—	—	—	—
	w/o CAP	0.78	14.71	5.60	0.38
	w/o AVP	0.83	18.51	5.02	0.27
	Ours	0.90	14.50	6.02	0.41
Freiburg world, 35 pedestrians	w/o NavHR	0.66	20.41	7.34	0.36
	w/o FMR	—	—	—	—
	w/o CAP	0.66	21.68	8.15	0.38
	w/o AVP	0.62	28.23	6.98	0.25
	Ours	0.74	18.66	7.21	0.38
Lobby world, 45 pedestrians	w/o NavHR	0.70	18.41	7.32	0.40
	w/o FMR	—	—	—	—
	w/o CAP	0.77	12.78	5.77	0.45
	w/o AVP	0.65	19.24	5.09	0.26
	Ours	0.85	14.8	5.97	0.40
Lobby world, 55 pedestrians	w/o NavHR	0.72	15.12	6.18	0.41
	w/o FMR	—	—	—	—
	w/o CAP	0.74	14.07	5.70	0.40
	w/o AVP	0.67	19.33	5.10	0.26
	Ours	0.78	14.10	5.75	0.41

Fig. 10(a) presents the reward curves for four ablated reward functions, while Fig. 10(b) illustrates the results of the ablation study on the attention module within the network. All curves have been smoothed using a moving average with a window size of 80 to reduce fluctuations and highlight overall trends. In Fig. 10(a), the w/o FMR curve remains consistently negative, which explains why the robot continues to rotate in place under this condition. The other three reward functions

show a decreasing trend in the later stages of training, particularly w/o NavHR, indicating that removing this module significantly impairs the learning efficiency and convergence of the policy. In Fig. 10(b), the reward curves for both attention modules exhibit a downward trend in the later stages of training, with the CBAM module showing a particularly pronounced decline. This indicates that, in the absence or replacement of the attention module, the policy struggles to



(a)



(b)

Fig. 10 The analysis of policy reward convergence under module ablations



Fig. 11 The mobile robot platforms and the LiDAR models used in experiments

effectively leverage critical features during the later stages of learning, leading to a deterioration in overall policy performance.

4.3 Hardware results

We validated the feasibility of the proposed algorithm in various real-world environments, including two indoor and two outdoor scenarios. In these settings, dense pedestrian traffic moved freely and interacted naturally with the robots. To evaluate the generalizability and applicability of the algorithm, experiments were carried out on three different types of mobile robot platform, with their configurations shown in Fig. 11.

The robot platform used for indoor experiments was a TurtleBot3 Waffle Pi equipped with an RPLIDAR A2, which delivers 360° 2D scans at up to 15 Hz with a maximum range of 12 meters, supporting obstacle detection and environment mapping. Outdoor experiments were carried out using the Autolabor mobile robot platform, equipped with a MID360 LiDAR, and the SCOUT 2.0 mobile robot platform, fitted with an RS-Helios-16P LiDAR. The experiments were carried out in a semistructured outdoor environment to assess

the reliability of obstacle detection. Both platforms followed predefined trajectories to ensure a fair comparison, and data from multiple runs were collected to account for environmental variability and sensor noise.

To facilitate a faithful sim-to-real evaluation, the real-world environments were meticulously reconstructed in Gazebo. As illustrated in Fig. 12, both the lobby and the laboratory scenes were modeled, capturing their spatial layouts, obstacles, and pedestrian flow patterns. Pedestrian density and navigation tasks were kept consistent between simulation and reality. By aligning these critical factors, we minimized discrepancies arising from domain differences and ensured comparable experimental conditions. The real-world robot directly deployed the policy trained in the simulation environment. For each environment, we defined 20 distinct navigation routes and the robot was deployed to traverse each route twice. This setup ensured sufficient coverage of the varying paths and interactions with pedestrians, allowing a robust evaluation of navigation performance. The repeated tests also captured variability due to stochastic pedestrian motion and minor environmental differences, providing more reliable performance statistics for both simulated and real-world scenarios.



Fig. 12 The Gazebo simulation environment is constructed to replicate the actual scene, including the lobby and the laboratory

Table 9 The comparison of robot navigation performance in real and simulated environments

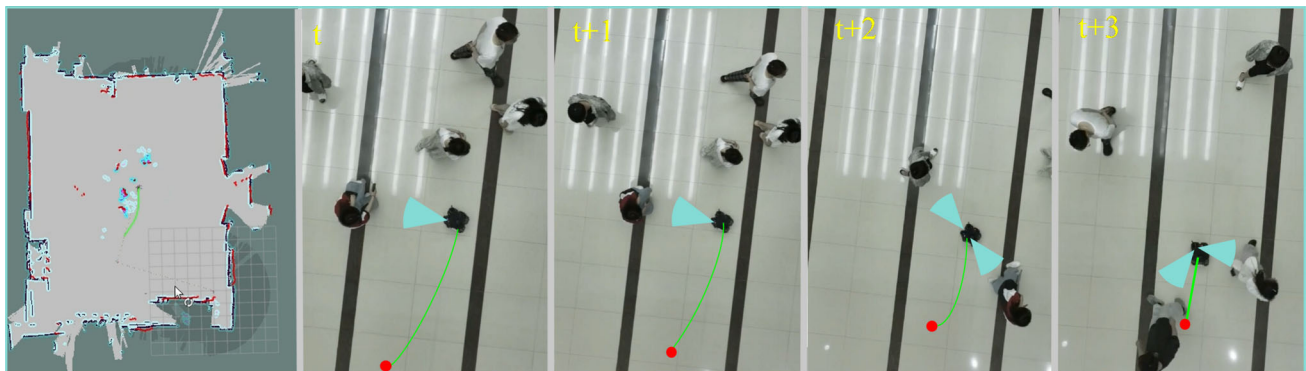
Environment	Success Rate $\uparrow$	Average Time (s) $\downarrow$	Average Length (m) $\downarrow$	Average Speed (m/s) $\uparrow$
Real laboratory world, 10 pedestrians	0.78	11.24	4.3	0.38
Simulated laboratory world, 10 pedestrians	0.83	12.40	4.80	0.39
Real lobby world, 15 pedestrians	0.93	15.80	6.45	0.41
Simulated lobby world, 15 pedestrians	1.00	15.36	6.60	0.43

In the laboratory environment, there are many small static obstacles, such as table legs, which are easily missed by the robot's sensors, directly affecting navigation performance. As shown in Table 9, the success rate in the real laboratory is lower than in the simulated environment, and the average speed is slightly lower, possibly due to occasional collisions or fine-tuned obstacle avoidance. Overall, the simulated environment is relatively idealized, overestimating the success rate and speed, while small obstacles and sensor errors in the real environment are the main sources of difference. In the real lobby environment, half of the walls are made of transparent glass, which can affect the precision of the LiDAR. The success rate in the real lobby is as high as 0.90, and the average time and path length differ little from the simulation, with only the average speed slightly lower. This indicates that our model has high navigation robustness in structured environments.

As shown in Fig. 13, we conducted obstacle avoidance tests in a lobby environment. The experimental setup involved ten pedestrians moving along irregular trajectories around the robot to simulate the complexity of real-world crowd interactions. Observations revealed that when pedestrians were either far from the robot or closely followed behind it, the robot maintained its pre-planned trajectory

without significant path adjustments. However, when a pedestrian approaches the robot, the robot quickly identified the potential collision risk and initiated evident avoidance maneuvers, successfully navigating around the pedestrian by adjusting their direction. This behavior demonstrates strong local obstacle avoidance capabilities.

Figure 14 illustrates the obstacle avoidance and navigation performance of the robot in a complex laboratory environment. The left side shows the corresponding global occupancy grid map, where static obstacles, such as desks, chairs, and partitions, are randomly distributed throughout the area. This setup forms a typical unstructured indoor environment, imposing stringent requirements on local planning and obstacle avoidance strategies. In this setting, the robot was guided by continuous autonomous navigation under the combined interference of dynamic and static pedestrians. The top row subplots show the robot avoiding obstacles with our algorithm in a dense crowd. When confronted with randomly moving pedestrians, the robot, guided by real-time perception and policy inference, was able to navigate flexibly within a limited safe distance, demonstrating strong trajectory optimization capabilities and environmental adaptability, effectively mitigating the collision risks caused by local congestion. The second row shows the

**Fig. 13** The analysis of robot avoidance behavior in a lobby environment under dense pedestrian conditions at different time periods, where the blue area denotes the obstacle directional range

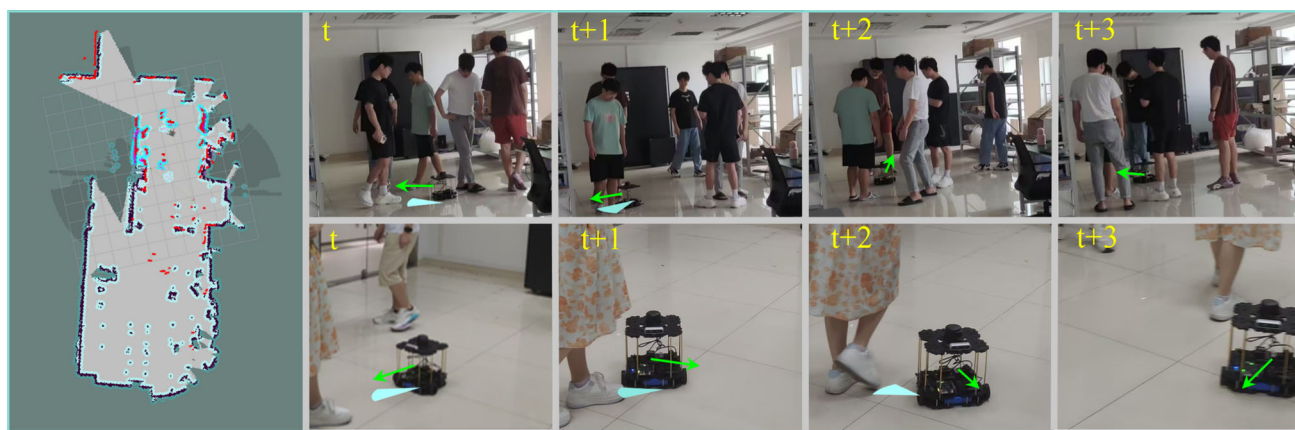


Fig. 14 The robot reaction to a high-density moving pedestrian at different times in a confined indoor environment. The green arrow denotes the robot's direction of motion, whereas the blue sector indicates the angular range of detected obstacles during navigation

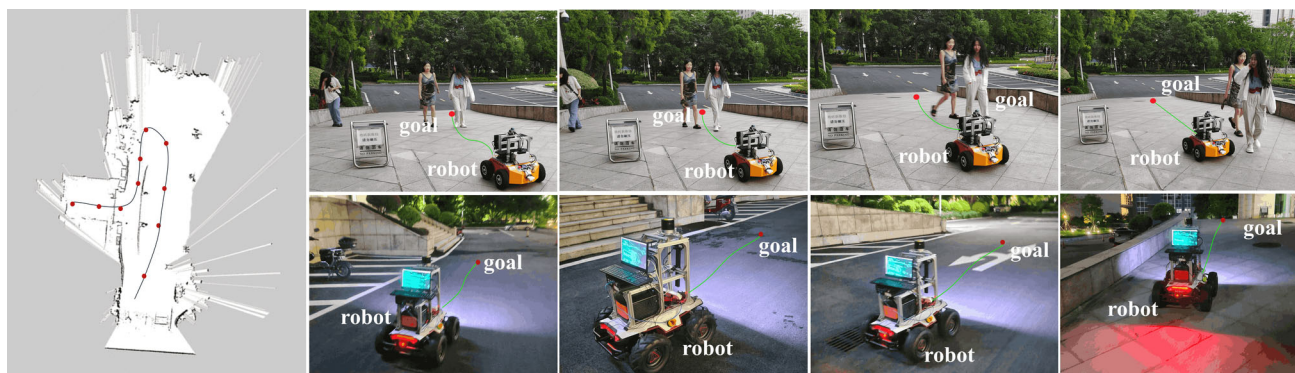


Fig. 15 The performance of the proposed algorithm in outdoor slope scenarios across daytime and nighttime tests

Table 10 The performance of different robots in slope scenario experiments

Hardware	Success Rate $\uparrow$	Average Time (s) $\downarrow$	Average Length (m) $\downarrow$	Average Speed (m/s) $\uparrow$
Autolabor	0.90	12.8	5.76	0.45
Scout 2.0	0.95	6.40	0.65	1.01

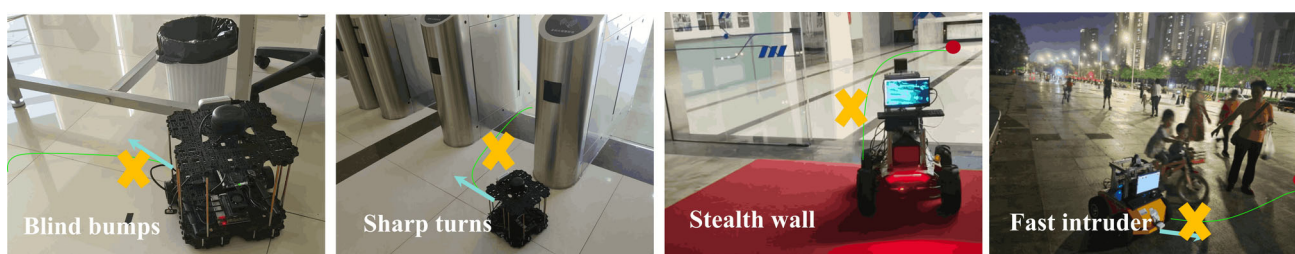


Fig. 16 The navigation failure scenarios in different environments

complete sequence of the robot dynamic obstacle avoidance maneuvers. Upon detecting pedestrians entering their local decision-making domain, the robot executes rapid and substantial in-place rotations to proactively adjust its heading, subsequently following a newly generated avoidance trajectory, and promptly returning to the optimal path toward the target. Throughout the entire process, the robot exhibited excellent motion coherence, low response latency, and strong policy generalization ability, validating the effectiveness and robustness of the proposed deep-reinforcement learning-based method in complex and dynamic environments.

As illustrated in Fig. 15, we tested the proposed algorithm in outdoor slope scenarios during both day and night. The daytime experiments were conducted on the Autolabor robot, while the nighttime experiments were conducted on the Scout 2.0 robot. The map contains the slope scene outside the glass door and the lobby scene inside. The red circles indicate target points, and the black lines represent test paths, one of which requires passing through a dynamic glass door. In these experiments, the robots demonstrated reliable location and trajectory planning on inclined terrain, achieving smooth and consistent navigation during the ascent and descent phases. When encountering pedestrians on the slope, the robots were able to avoid dynamic obstacles with high precision, adjusting their trajectories in real time. As shown in Table 10, both robots successfully completed most of the 20 navigation tasks, performing well at low and high speeds. However, in the glass door area, the Scout2.0 robot succeeded once during the short opening of the door but failed in another attempt, while the Autolabor robot failed. These results indicate that the proposed algorithm performs well in outdoor navigation and dynamic obstacle avoidance, but its performance is limited in rapidly changing environments.

The proposed algorithm excels in dynamic navigation due to its deep reinforcement learning framework, which enables real-time policy inference and generalization across diverse environments. By integrating high-resolution LiDAR perception with a closed-loop decision-making mechanism, the system ensures accurate mapping, timely obstacle avoidance, and smooth trajectory planning. Its policy network, shaped by a well-designed reward function, balances safety, efficiency, and goal direction, allowing the robot to respond adaptively to complex and unstructured scenarios. These mechanisms collectively ensure robust, efficient, and reliable performance in real-world tasks.

4.4 Failure cases and analysis

In the simulation environment, failures were most pronounced during navigation in dense crowds, where the robot frequently became surrounded and was unable to identify feasible directions or viable escape paths. Its perception module was limited to a forward-facing field of view, which resulted

in the inability to account for pedestrians approaching from behind. Additional failures occurred at sharp corners, where the restricted turning radius hindered successful maneuvers, and on narrow pathways, where interactions with pedestrians often led to near-collision events. These shortcomings can be primarily attributed to three factors: (1) the restricted perceptual range of the robot, (2) the absence of backward motion capability, which reduced safety in extreme situations, and (3) the insufficient robustness of the navigation algorithm, which had been trained on relatively simplified scenarios.

In contrast, the real-world experiments revealed a broader spectrum of failure cases, many of which could not be reproduced in simulation. As shown in Fig. 16, transparent obstacles such as glass doors created ‘invisible wall’ scenarios, in which LiDAR-based detection failed to provide accurate distance measurements. Small-scale objects, including cables and doorstoppers, were frequently overlooked due to limited sensor resolution. Furthermore, the sudden appearance of fast-moving pedestrians posed significant challenges to the perception–prediction pipeline, often resulting in delayed or unsafe responses.

The observed failure cases can be mitigated by strengthening both perception and navigation capabilities. In particular, enhancing the sensory system with multimodal and omnidirectional sensing facilitates robust detection of small-scale obstacles. Currently, navigation algorithms can be trained in heterogeneous, densely populated, and dynamic environments while incorporating robust planning methods that account for sensor noise and environmental uncertainties. Furthermore, bridging the sim-to-real gap requires realistic simulation, domain randomization, and iterative refinement informed by real-world observations to capture critical failure modes effectively. In practical engineering applications, a fault-tolerant safety mechanism can be incorporated. This mechanism overrides the learned navigation policy upon detecting a potential collision, prompting the robot to halt and perform a rotational search until a collision-free space is identified. Conceptually, this approach is analogous to the stop and wait strategy employed in the A1-RD policy, providing an additional layer of safety without modifying the primary navigation algorithm.

5 Conclusion

In this study, we propose a reinforcement learning framework that takes advantage of LiDAR-based perception of the surrounding environment to allow autonomous ground vehicles to navigate safely and efficiently in complex real-world scenarios. The proposed obstacle avoidance strategy ensures reliable and smooth navigation in dynamic crowded settings by using only a single LiDAR sensor. This framework improves the robustness of the policy to environmental

variability by encoding all perceptual inputs in the robot's local coordinate frame. In addition, an adaptive directional reward function was designed to guide policy training, effectively encouraging the robot to select safer and more efficient trajectories in dynamic environments. Our method outperforms several state-of-the-art navigation algorithms, demonstrating strong generalization to varying crowd densities and unseen environments. Ablation and extended studies confirm the effectiveness of each module and the robustness of the overall system. Hardware experiments further show that the approach reliably operates across diverse indoor and outdoor scenarios, bridging the sim-to-real gap without any fine-tuning or modification. We also analyzed and discussed negative cases to better understand the limitations of our approach. These analyzes revealed specific failure modes, such as difficulties in navigating dense crowds, narrow passages, and unexpected dynamic obstacles. The insights gained from these cases inform potential improvements in perception, motion planning, and algorithmic robustness for future work.

5.1 Limitations and future work

Although the proposed algorithm demonstrates satisfactory navigation performance in simulation environments, several challenges remain when deploying it in real world scenarios. Our experiments reveal that transparent glass walls can interfere with LiDAR perception, resulting in incomplete or inaccurate obstacle mapping. Furthermore, due to sensor limitations and environmental noise, small static obstacles that are reliably detected in simulation may be missed under real-world conditions. These factors contribute to the sim-to-real gap observed in our experiments.

To address these challenges, our future work will focus on integrating sensor fusion techniques, improving obstacle detection algorithms, and collecting more diverse real-world datasets to better bridge the gap between simulation and reality. Efforts will also be directed toward optimizing real-time performance in large-scale and densely populated environments. Although the current model emphasizes representational capacity and achieves strong navigation performance, this advantage comes at the cost of increased computational complexity. We therefore plan to explore model compression and knowledge distillation methods to improve the efficiency of the framework for embedded or low-power robotic platforms. In addition, we intend to investigate multi-agent coordination strategies for safe and efficient collaborative navigation in shared dynamic spaces. Finally, our objective is to improve sim-to-real transfer through various training scenarios and iterative real-world validation, further improving the safety, robustness, and operational reliability of the proposed approach.

Acknowledgements This research was supported by the International Cooperation Fund of Jilin Province (Grant No. 20210402074GH), the Public Welfare Science and Technology Program of Zhongshan City (2023B2010), the Innovative Research Team Grant (CXTD2023002), and a research grant from Naif Arab University for Security Sciences (NAUSS-23-R26).

Author Contributions Conceptualization, Jin Meng; Methodology, Jin Meng; Software, Jin Meng and Aakash Kumar; Validation, Jin Meng; Formal analysis, Jin Meng and Jiahui Zou; Investigation, Jiahui Zou; Data curation, Ruibing Yang; Writing—original draft, Jin Meng and Jiahui Zou; Writing—review & editing, Shifeng Wang and Jonghyuk Kim; Supervision, Jin Meng, Ruibing Yang and Aakash Kumar.

Data Availability Data will be made available upon request.

Declarations

Competing Interests The authors declare that they have no competing interests.

Open Access This article is licensed under a Creative Commons Attribution-NonCommercial-NoDerivatives 4.0 International License, which permits any non-commercial use, sharing, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if you modified the licensed material. You do not have permission under this licence to share adapted material derived from this article or parts of it. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit <http://creativecommons.org/licenses/by-nc-nd/4.0/>.

References

- Alaliyat S, Oucheikh R, Hameed I (2019) Path planning in dynamic environment using particle swarm optimization algorithm. In: 2019 8th International Conference on Modeling Simulation and Applied Optimization (ICMSAO) pp 1–5. <https://doi.org/10.1109/ICMSAO.2019.8880434>
- Asadi K, Suresh AK, Ender A et al (2020) An integrated ugv-uav system for construction site data collection. *Autom Constr* 112:103068. <https://doi.org/10.1016/j.autcon.2019.103068>
- Atila Ü, Uçar M, Akyol K et al (2021) Plant leaf disease classification using efficientnet deep learning model. *Eco Inform* 61:101182. <https://doi.org/10.1016/j.ecoinf.2020.101182>
- Bellman R (1957) A markovian decision process. *Journal of mathematics and mechanics* pp 679–684. <https://doi.org/10.1512/iumj.1957.6.56038>
- Borenstein J, Koren Y (1989) Real-time obstacle avoidance for fast mobile robots. *IEEE Trans Syst Man Cybern* 19(5):1179–1187. <https://doi.org/10.1109/robot.1990.126042>
- Boutilier C (2002) A pomdp formulation of preference elicitation problems. *AAAI/IAAI* pp 239–246. <https://dl.acm.org/doi/10.5555/777092.777132>
- Camara F, Bellotto N, Cosar S et al (2020) Pedestrian models for autonomous driving part ii: high-level models of human behav-

- ior. *IEEE Trans Intell Transp Syst* 22(9):5453–5472. <https://doi.org/10.1109/TITS.2020.3006767>
- Cimurs R, Suh IH, Lee JH (2022) Goal-driven autonomous exploration through deep reinforcement learning. *IEEE Robot Autom Lett* 7(2):730–737. <https://doi.org/10.1109/LRA.2021.3133591>
- Desouza GN, Kak AC (2002) Vision for mobile robot navigation: a survey. *IEEE Trans Pattern Anal Mach Intell* 24(2):237–267. <https://doi.org/10.1109/34.982903>
- Fan T, Long P, Liu W et al (2020) Distributed multi-robot collision avoidance via deep reinforcement learning for navigation in complex scenarios. *Int J Robot Res* 39(7):856–892. <https://doi.org/10.1177/0278364920916531>
- Fox D, Burgard W, Thrun S (1997) The dynamic window approach to collision avoidance. *IEEE Robot Autom Mag* 4(1):23–33. <https://doi.org/10.1109/100.580977>
- Guldenring R, Görner M, Hendrich N et al (2020) Learning local planners for human-aware navigation in indoor environments. In: 2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) pp 6053–6060. <https://doi.org/10.1109/iros45743.2020.9341783>
- Juang CF, Chang YC (2011) Evolutionary-group-based particle-swarm-optimized fuzzy controller with application to mobile-robot navigation in unknown environments. *IEEE Trans Fuzzy Syst* 19(2):379–392. <https://doi.org/10.1109/tfuzz.2011.2104364>
- Kennedy J, Eberhart R (1995) Particle swarm optimization. In: *Proceedings of ICNN'95 - International conference on neural networks vol 4*, pp 1942–1948. <https://doi.org/10.1109/ICNN.1995.488968>
- Kim M, Lee D, Ahn J et al (2021) Model predictive control method for autonomous vehicles using time-varying and non-uniformly spaced horizon. *IEEE Access* 9:86475–86487. <https://doi.org/10.1109/access.2021.3088937>
- Kiran BR, Sobh I, Talpaert V et al (2021) Deep reinforcement learning for autonomous driving: a survey. *IEEE Trans Intell Transp Syst* 23(6):4909–4926. <https://doi.org/10.1109/TITS.2021.3054625>
- Larson J, Bruch M, Ebken J (2006) Autonomous navigation and obstacle avoidance for unmanned surface vehicles. *Unmanned Syst Technol* VIII 6230:53–64. <https://doi.org/10.1117/12.663798>
- Li Y, Hu X, Zhuang Y et al (2019) Deep reinforcement learning (drl): another perspective for unsupervised wireless localization. *IEEE Internet Things J* 7(7):6279–6287. <https://doi.org/10.1109/jiot.2019.2957778>
- Li W, Yue M, Shanguan J et al (2023) Navigation of mobile robots based on deep reinforcement learning: reward function optimization and knowledge transfer. *Int J Control Autom Syst* 21(2):563–574. <https://doi.org/10.1007/s12555-021-0642-7>
- Li Y, Ji Y, Zhang L et al (2024) Path planning method and control of mobile robot with uncertain dynamics based on improved artificial potential fields. In: 2024 IEEE 14th International Conference on CYBER Technology in Automation, Control, and Intelligent Systems (CYBER) pp 523–528. <https://doi.org/10.1109/cyber63482.2024.10748726>
- Liu W, Zhang B, Liu P et al (2024) Velocity obstacle guided motion planning method in dynamic environments. *J King Saud Univ-Comp Inf Sci* 36(1):101889. <https://doi.org/10.1016/j.jksuci.2023.101889>
- Long P, Fan T, Liao X et al (2018) Towards optimally decentralized multi-robot collision avoidance via deep reinforcement learning. In: 2018 IEEE International Conference on Robotics and Automation (ICRA) pp 6252–6259. <https://doi.org/10.1109/icra.2018.8461113>
- Maoudj A, Hentout A (2020) Optimal path planning approach based on q-learning algorithm for mobile robots. *Appl Soft Comput* 97:106796. <https://doi.org/10.1016/j.asoc.2020.106796>
- Missura M, Bennewitz M (2019) Predictive collision avoidance for the dynamic window approach. In: 2019 International Conference on Robotics and Automation (ICRA) pp 8620–8626. <https://doi.org/10.1109/icra.2019.8794386>
- Nguyen MN, Van M, McIlvanna S et al (2024) Model-free safety critical model predictive control for mobile robot in dynamic environments. *IEEE Trans Intell Veh*. <https://doi.org/10.1109/tiv.2024.3389111>
- Ouyang D, He S, Zhang G et al (2023) Efficient multi-scale attention module with cross-spatial learning. In: 2023 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP) pp 1–5. <https://doi.org/10.1109/icassp49357.2023.10096516>
- Qin SJ, Badgwell TA (2003) A survey of industrial model predictive control technology. *Control Eng Pract* 11(7):733–764. [https://doi.org/10.1016/S0967-0661\(02\)00186-7](https://doi.org/10.1016/S0967-0661(02)00186-7)
- Rehani M (2015) Limitations of diagnostic reference level (drl) and introduction of acceptable quality dose (aqd). *Br J Radiol* 88(1045):20140344. <https://doi.org/10.1259/bjr.20140344>
- Rösmann C, Makarow A, Bertram T (2021) Online motion planning based on nonlinear model predictive control with non-euclidean rotation groups. In: 2021 European Control Conference (ECC) pp 1583–1590. <https://doi.org/10.23919/ECC54610.2021.9654872>
- Rusu AA, Večerík M, Rothörl T et al (2017) Sim-to-real robot learning from pixels with progressive nets. *Conference on robot learning* pp 262–270. <https://doi.org/10.48550/arXiv.1610.04286>
- Sarkar R, Barman D, Chowdhury N (2022) Domain knowledge based genetic algorithms for mobile robot path planning having single and multiple targets. *J King Saud Univ- Comput Inf Sci* 34(7):4269–4283. <https://doi.org/10.1016/j.jksuci.2020.10.010>
- Schwenzer M, Ay M, Bergs T et al (2021) Review on model predictive control: an engineering perspective. *Int J Adv Manuf Technol* 117(5):1327–1349. <https://doi.org/10.1007/s00170-021-07682-3>
- Tao B, Kim JH (2024) Deep reinforcement learning-based local path planning in dynamic environments for mobile robot. *J King Saud Univ Comput Inf Sci* 36(10):102254. <https://doi.org/10.1016/j.jksuci.2024.102254>
- Woo S, Park J, Lee JY et al (2018) Cbam: Convolutional block attention module. In: *Proceedings of the European conference on computer vision (ECCV)* pp 3–19. https://doi.org/10.1007/978-3-030-01234-2_1
- Xie Z, Dames P (2023) Drl-vo: Learning to navigate through crowded dynamic scenes using velocity obstacles. *IEEE Trans Rob* 39(4):2700–2719. <https://doi.org/10.1109/tro.2023.3257549>
- Yu C, Velu A, Vinitsky E et al (2022) The surprising effectiveness of ppo in cooperative multi-agent games. *Adv Neural Inf Process Syst* 35:24611–24624. <https://doi.org/10.48550/arXiv.2103.01955>